**«Л. Н. ГУМИЛЕВ АТЫНДАҒЫ ЕУРАЗИЯ ҰЛТТЫҚ УНИВЕРСИТЕТІ»**
**КеАҚ**

**ЖАСАНДЫ ИНТЕЛЛЕКТ ТЕХНОЛОГИЯЛАРЫ**
**КАФЕДРАСЫ**

# ДЕРЕКТЕР БАЗАСЫ ЖӘНЕ SQL СҰРАТЫМДАР ПӘНІ БОЙЫНША КУРСТЫҚ ЖҰМЫС

**ТАҚЫРЫБЫ:** «Мемлекеттік жоғары оқу орнындағы оқу үдерісіне қатысты құжаттарды есепке алуға арналған деректер базасын құру»

Орындағандар: Ербол Елқұт,
Авьдал Бахатбек
6В06112 – «Жасанды интеллект технологиялары»
ББ 2-курс студенттері

Ғылыми жетекші: Елибаева Г.К.
магистр, аға оқытушы

Астана
2026

**«Л. Н. ГУМИЛЕВ АТЫНДАҒЫ ЕУРАЗИЯ ҰЛТТЫҚ УНИВЕРСИТЕТІ» КеАҚ**

Кафедра Жасанды интеллект технологиялары
Пән Деректер базасы және SQL сұратымдар

БЕКІТЕМІН
Кафедра меңгерушісі
_____
«____»_____20__ ж.

**Білім алушының курстық жұмысына**
**ТАПСЫРМА**

**Білім алушылар:** Ербол Елқұт, Авьдал Бахатбек, 6В06112 – «Жасанды интеллект технологиялары» ББББ, 2-курс

**Курстық жұмыс тақырыбы:** Мемлекеттік жоғары оқу орнындағы оқу үдерісіне қатысты құжаттарды есепке алуға арналған деректер базасын құру

**Мақсатты бағыт:** Python SQLite ДББЖ негізінде PyCharm ортасында ЖОО-ның оқу үдерісі құжаттарын тіркеуге, іздеуге және талдауға арналған реляциялық деректер базасы мен қосымшаны әзірлеу

**Курстық жұмыс көлемі:** 25–30 бет (қосымшаларсыз); бағдарламалық жоба (Python, SQLite); қорғау презентациясы

Курстық жұмысты әзірлеу барысы жайында жетекшіге баяндау мерзімі:
а) жинақталған мәлімет пен курстық жұмысты әзірлеу барысы жайында баяндау «___»_____20__ ж.
ә) курстық жұмысты жазу барысы жайында баяндау «___»_____20__ ж.
Курстық жұмысты тапсыру мерзімі «___»_____20__ ж.

Курстық жұмыс жетекшісі: _____ Елибаева Г.К.
«___»_____20__ ж.

**МАЗМҰНЫ**

КІРІСПЕ.....4
1 Мемлекеттік жоғары оқу орнындағы оқу үдерісіне қатысты құжаттарды есепке алуға арналған деректер базасын құрудың теориялық негіздері.....5
1.1 Деректер базасы теориясының негізгі ұғымдары.....5
1.2 Деректердің негізгі модельдері.....9
1.3 Деректерді жобалаудың негізгі кезеңдері.....11
2 Мемлекеттік жоғары оқу орнындағы оқу үдерісіне қатысты құжаттарды есепке алуға арналған деректер базасын құрудың практикалық негіздері.....13
2.1 Пәндік аймақ бойынша деректер базасының жалпы сипаттамасы.....13
2.2 Қосымша программасы және онымен жұмыс.....16
2.3 Деректер базасын PyCharm ортасында өңдеу жұмыстары.....20
3 Оқу үдерісі құжаттарын есепке алуға арналған деректер базасын зерттеу нәтижелері және қорытынды.....27
ҚОРЫТЫНДЫ.....29
ПАЙДАЛАНЫЛҒАН ДЕРЕК КӨЗДЕРІНІҢ ТІЗІМІ.....30
ҚОСЫМША А Деректер базасының схемасы (schema.sql).....31
ҚОСЫМША Ә Бағдарлама кодының негізгі фрагменттері.....33
ҚОСЫМША Б step04_update_delete.py скриптінің толық мәтіні.....35
ҚОСЫМША В Қосымша беттері және скрипттердің орындалу нәтижелері.....37
БЕЛГІЛЕУЛЕР МЕН ҚЫСҚАРТУЛАР.....41

# КІРІСПЕ

Жоғары оқу орнында (ЖОО) оқу үдерісі көптеген құжаттар арқылы жүреді. Силлабус пен оқу жұмыс жоспары не оқытылатынын көрсетеді, емтихан ведомосі бағаларды тіркейді, контингент бойынша бұйрықтар білім алушыны қабылдауды, курстан курсқа ауыстыруды немесе академиялық демалысқа жіберуді рәсімдейді, ал анықтамалар білім алушының сұранысы бойынша беріледі. Бұл құжаттарды кафедралар мен тіркеу офисі дайындайды, келіседі, бекітеді және мұрағатқа тапсырады.

Құжаттар туралы мәліметтер қағаз журналда немесе бөлек Excel файлдарында жүргізілсе, бірнеше қиындық туындайды: құжаттың қай кезеңде тұрғанын білу қиын, тіркеу нөмірлері қайталануы мүмкін, белгілі бір білім алушыға қатысты барлық бұйрықтар мен анықтамаларды табу көп уақыт алады. Сондықтан оқу құжаттарын бір жерде тіркеуге, іздеуге және сүзгілеуге мүмкіндік беретін деректер базасын құру – өзекті міндет.

**Жұмыстың мақсаты** – Python тілі мен SQLite деректер базасын басқару жүйесін (ДББЖ) пайдаланып, PyCharm ортасында ЖОО-дағы оқу үдерісіне қатысты құжаттарды есепке алуға арналған реляциялық деректер базасын және онымен жұмыс істейтін қарапайым веб-қосымшаны әзірлеу.

Мақсатқа жету үшін келесі **міндеттер** қойылды:

– деректер базасының негізгі ұғымдарын, деректер модельдерін және жобалау кезеңдерін оқып-үйрену;

– пәндік аймақты талдап, құжат түрлерін және құжаттың өмірлік циклін анықтау;

– ER-модель құрып, оны кестелерге түрлендіру;

– деректер базасын құру, оған қосылу және ажырату, кестелер мен олардың арасындағы байланыстарды құру, диаграмма салу;

– деректерді енгізу, өзгерту, жою, бір кестеден және бірнеше кестеден іріктеу, сүзгілеу, топтау және сұрыптау сұратымдарын жазу;

– әкімші мен қонақ рөлдері бар веб-қосымша жасап, оны тексеру.

**Зерттеу объектісі** – ЖОО-дағы оқу үдерісінің құжаттары. **Зерттеу пәні** – осы құжаттарды реляциялық деректер базасында сақтау және SQL сұратымдары арқылы өңдеу.

**Қолданылған құралдар:** Python 3.11 тілі және оның стандартты `sqlite3`, `http.server`, `hashlib` модульдері, SQLite ДББЖ, PyCharm әзірлеу ортасы, веб-интерфейс үшін HTML, CSS және JavaScript.

**Практикалық маңызы.** Жасалған деректер базасын кафедра немесе тіркеу офисі құжаттарды тіркеу журналы ретінде пайдалана алады: құжатты нөмірі немесе атауы бойынша табу, мәртебесін қадағалау, білім алушыға қатысты құжаттарды көру бір бетте орындалады. Жобадағы барлық аты-жөндер ойдан құрастырылған.

Курстық жұмыс кіріспеден, үш бөлімнен, қорытындыдан, пайдаланылған дерек көздерінің тізімінен, қосымшалардан және белгілеулер мен қысқартулар тізімінен тұрады.

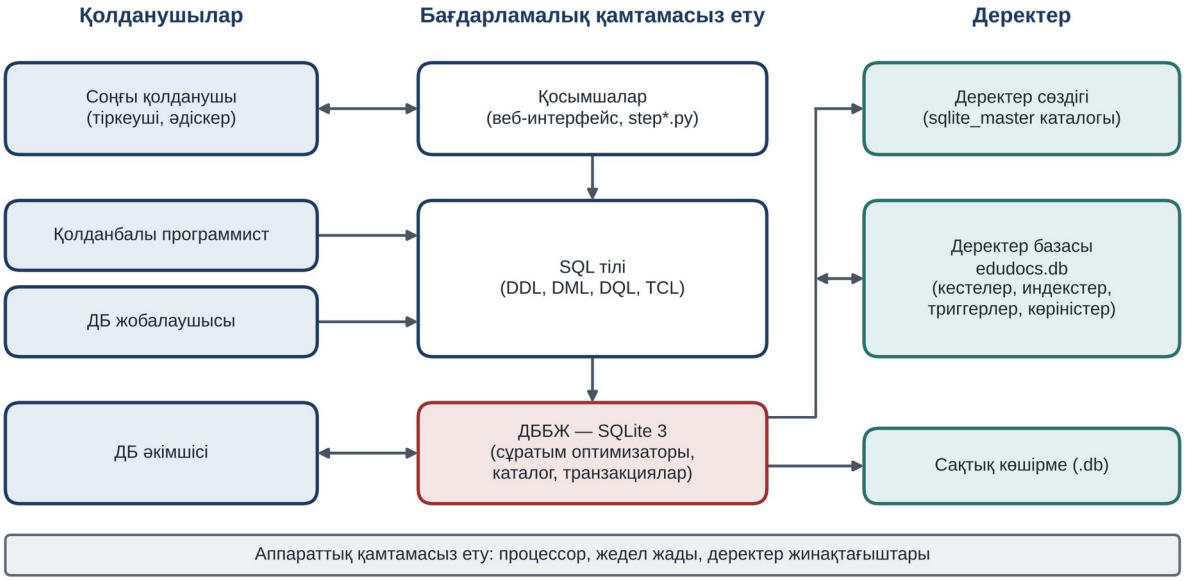
# 1 Мемлекеттік жоғары оқу орнындағы оқу үдерісіне қатысты құжаттарды есепке алуға арналған деректер базасын құрудың теориялық негіздері

## 1.1 Деректер базасы теориясының негізгі ұғымдары

Деректер базасы теориясында алдымен «дерек» пен «ақпарат» ұғымдарын ажырату қажет. **Дерек** – белгілі бір пішімде тіркелген, сақтауға, тасымалдауға және өңдеуге жарамды фактілер, сандар немесе мәтіндер. **Ақпарат** – деректерді өңдеу нәтижесінде алынған және адамға белгілі бір мағына беретін мәлімет [1]. Мысалы, «СИЛ-2026/001», «2026-09-01» және «бекітілді» – жеке деректер, ал «2026 жылы тіркелген бірінші силлабус 1 қыркүйекте бекітілді» деген тұжырым – ақпарат.

**Деректер базасы (ДБ)** – белгілі бір тақырыпқа (пәндік аймаққа) қатысты, белгілі бір ережемен ұйымдастырылған және компьютерде сақталатын өзара байланысқан деректер жиынтығы [2]. Мысалы, кафедраның барлық құжаттары туралы мәліметтер бір деректер базасында сақтала алады. Жақсы жобаланған деректер базасының негізгі қасиеттері: әр дерек бір рет сақталады және артық қайталанбайды; деректер бүтін және бір-біріне қайшы келмейді; бір деректерді бірнеше қолданушы мен бағдарлама бірге пайдалана алады; деректер оларды өңдейтін бағдарламаларға тәуелді болмайды; деректер рұқсатсыз қолжетімділіктен қорғалады [3; 4].

**Деректер базасын басқару жүйесі (ДББЖ)** – деректер базасын құруға, оған деректер енгізуге, өзгертуге, іздеуге және қорғауға арналған бағдарламалық кешен [5]. Белгілі ДББЖ-ларға MySQL, PostgreSQL, Microsoft SQL Server, Oracle және SQLite жатады. К. Дейттің пікірінше, деректер базасы жүйесі төрт бөліктен тұрады: деректер, аппараттық құралдар, бағдарламалық құралдар және қолданушылар [6]. Бұл бөліктердің осы жұмысқа қатысты байланысы 1-суретте көрсетілген.




1-сурет. Деректер базасы жүйесінің құрамы

ДББЖ атқаратын негізгі функциялар төмендегідей [5; 7]:

– деректерді анықтау – кестелердің, өрістердің және байланыстардың құрылымын сипаттау;

– деректерді өңдеу – жазбаларды енгізу, өзгерту, жою және іздеу;

– деректердің бүтіндігін қамтамасыз ету – енгізілген мәндердің шектеулерге сәйкестігін тексеру;

– бірнеше қолданушының қатар жұмысын басқару және транзакцияларды орындау;

– деректерді қорғау – қолданушыларды тіркеу және оларға құқық беру;

– сақтық көшірме жасау және ақаулықтан кейін деректерді қалпына келтіру.

Деректер базасымен жұмыс істейтін адамдар бірнеше топқа бөлінеді. Деректер базасын жобалаушы кестелердің құрылымын анықтайды, қолданбалы программист деректер базасымен жұмыс істейтін бағдарлама жазады, ДБ әкімшісі жүйенің дұрыс жұмыс істеуін, қауіпсіздігін және сақтық көшірмелерін қадағалайды, ал соңғы қолданушы (мысалы, тіркеуші немесе әдіскер) дайын бағдарлама арқылы деректерді енгізеді және іздейді [2].

ANSI/SPARC комитеті ұсынған архитектура бойынша деректер базасы үш деңгейде сипатталады [6; 8]. **Сыртқы деңгей** – жеке қолданушы немесе қолданушылар тобы көретін деректер бөлігі; мысалы, біздің жобада қонақ тек бекітілген құжаттарды көреді. **Концептуалдық деңгей** – бүкіл деректер базасының логикалық құрылымы: кестелер, өрістер, байланыстар және шектеулер. **Ішкі деңгей** – деректердің дискіде физикалық сақталу тәсілі: файлдар, беттер және индекстер. Деңгейлердің бөлінуі **деректердің тәуелсіздігін** қамтамасыз етеді. Логикалық тәуелсіздік – концептуалдық схеманы өзгерткенде (мысалы, кестеге жаңа өріс қосқанда) сыртқы деңгейдегі бағдарламаларды өзгертудің қажеті болмауы. Физикалық тәуелсіздік – деректерді сақтау тәсілін өзгерткенде (мысалы, индекс қосқанда) концептуалдық схеманың өзгермеуі.

Реляциялық модельдің негізгі ұғымы – **қатынас** (relation), ол кесте түрінде көрсетіледі. Қатынастың жолы **кортеж** (жазба), ал бағаны **атрибут** (өріс) деп аталады. Атрибут қабылдай алатын мүмкін мәндердің жиыны **домен** деп аталады; мысалы, `course` өрісінің домені – 1-ден 4-ке дейінгі бүтін сандар. Қатынастағы атрибуттар саны оның **дәрежесі**, ал кортеждер саны **қуаттылығы** деп аталады [6; 9]. Қатынастың қасиеттері: кортеждер қайталанбайды, кортеждер мен атрибуттардың реті маңызды емес, әр атрибуттың мәні атомарлы, яғни бөлінбейтін болады. Реляциялық теорияның терминдері, олардың кестелік баламалары және жобадағы мысалдары 1-кестеде берілген.

Кесте-1

Реляциялық модельдің негізгі терминдері және олардың жобадағы мысалдары

| **Реляциялық термин** | **Кестелік термин** | **EduDocs жобасындағы мысал** |
|---|---|---|
| Қатынас | кесте | `documents` – құжаттар кестесі |
| Кортеж | жазба (жол) | СИЛ-2026/001 силлабусы туралы жол |
| Атрибут | өріс (баған) | `doc_date` – құжаттың күні |
| Домен | өрістің рұқсат етілген мәндері | `course`: 1, 2, 3, 4 |
| Дәреже | өрістер саны | `documents` кестесінде 10 өріс |
| Қуаттылық | жазбалар саны | `documents` кестесінде 36 жазба |
| Бастапқы кілт | PRIMARY KEY | `documents.id` |
| Сыртқы кілт | FOREIGN KEY | `documents.type_id` → `document_types.id` |

Кестедегі жазбаларды бір-бірінен ажырату үшін **кілттер** қолданылады. Кортежді бірмәнді анықтайтын атрибуттар жиыны **суперкілт** деп аталады. Артық атрибуты жоқ, яғни бірде-бір атрибутын алып тастауға болмайтын суперкілт – **кандидат кілт** (ықтимал кілт). Кандидат кілттердің бірі **бастапқы кілт** (PRIMARY KEY) ретінде таңдалады, қалғандары **балама кілт** болып қалады; SQL тілінде олар UNIQUE шектеуімен жарияланады. Бірнеше өрістен тұратын кілт **құрама кілт** деп аталады. **Сыртқы кілт** (FOREIGN KEY) – басқа кестенің бастапқы кілтіне сілтеме жасайтын өріс; кестелер дәл осы кілт арқылы байланысады [9]. Мысалы, `students` кестесінде `id` – бастапқы кілт, `record_book` – балама кілт, ал `document_students` кестесінде (`document_id`, `student_id`) жұбы құрама кілт болып табылады.

Деректердің дұрыстығын **бүтіндік шектеулері** қамтамасыз етеді. Олардың үш негізгі түрі бар [8]. **Мәннің бүтіндігі**: бастапқы кілт бос (NULL) болмауы және қайталанбауы тиіс. **Сілтемелік бүтіндік**: сыртқы кілттің мәні сілтеме жасалған кестеде шынымен бар болуы тиіс. Сілтеме жасалған жазба жойылғанда не болатынын ереже анықтайды: RESTRICT – жоюға тыйым салады, CASCADE – тәуелді жазбаларды да жояды, SET NULL – сыртқы кілтке бос мән жазады. **Домендік бүтіндік**: өрістің мәні рұқсат етілген жиыннан алынуы керек; ол өрістің типімен және NOT NULL, UNIQUE, CHECK, DEFAULT шектеулерімен тексеріледі. Мысалы, CHECK шектеуі құжаттың мәртебесі тек «жоба», «келісуде», «бекітілді» немесе «мұрағатта» болуын талап етеді.

Деректер базасымен жұмыс **SQL** (Structured Query Language) тілі арқылы жүргізіледі. SQL – реляциялық деректер базаларына арналған стандартталған декларативтік тіл: пайдаланушы нәтижені қалай алу керегін емес, қандай нәтиже керек екенін көрсетеді. Тілдің алғашқы нұсқасы 1970 жылдары IBM компаниясында жасалып, кейін ANSI және ISO стандарттарына айналды [10]. SQL командалары атқаратын қызметіне қарай бірнеше топқа бөлінеді (2-кесте).

Кесте-2

SQL командаларының топтары

| Топ | Мақсаты | Негізгі командалар |
|---|---|---|
| DDL (Data Definition Language) | деректер құрылымын анықтау | CREATE, ALTER, DROP |
| DML (Data Manipulation Language) | деректерді енгізу, өзгерту, жою | INSERT, UPDATE, DELETE |
| DQL (Data Query Language) | деректерді іріктеу | SELECT |
| DCL (Data Control Language) | қолжетімділікті басқару | GRANT, REVOKE |
| TCL (Transaction Control Language) | транзакцияларды басқару | BEGIN, COMMIT, ROLLBACK |

SQLite-та қолжетімділікті басқару (DCL) командалары жоқ, өйткені деректер базасы бір файлда сақталады және серверсіз жұмыс істейді; қолданушылардың құқығын бағдарламаның өзі тексереді. SELECT сұратымының жалпы құрылымы: SELECT – шығарылатын өрістер, FROM – кестелер, WHERE – жолдарды сүзу шарты, GROUP BY – топтау, HAVING – топтарды сүзу, ORDER BY – сұрыптау. Сұратым осы ретпен жазылғанымен, ДББЖ оны логикалық тұрғыдан басқа ретпен өңдейді: алдымен FROM және WHERE, содан кейін GROUP BY мен HAVING, соңында SELECT пен ORDER BY [2].

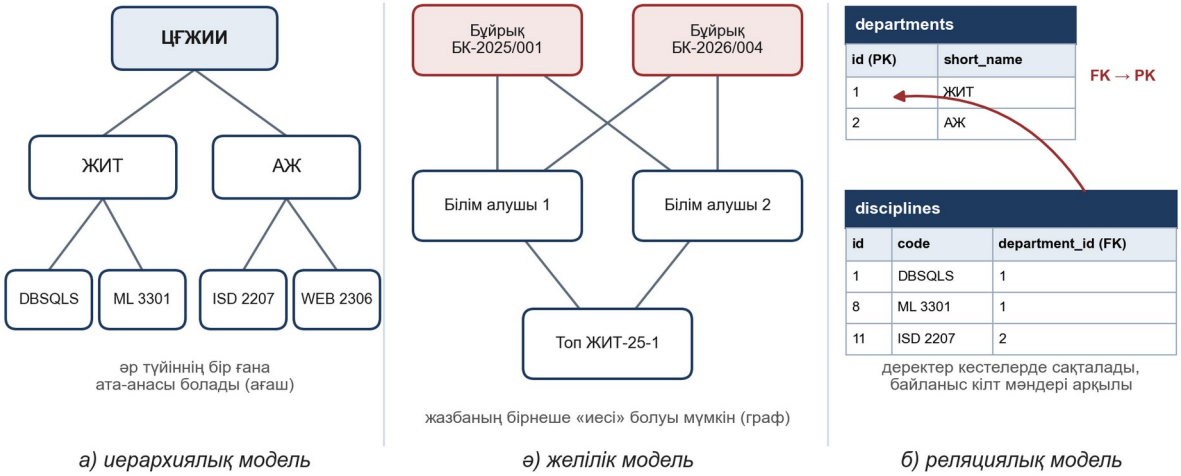
**Транзакция** – бір бүтін ретінде орындалатын әрекеттер тізбегі. Транзакция ACID деп аталатын төрт қасиетке ие болуы керек [8]: **атомарлық** – транзакцияның барлық әрекеті орындалады немесе ешқайсысы орындалмайды; **келісімділік** – транзакция базаны бір дұрыс күйден екінші дұрыс күйге ауыстырады; **оқшаулық** – қатар орындалатын транзакциялар бір-біріне әсер етпейді; **беріктік** – COMMIT командасынан кейін өзгерістер ақаулық болса да сақталады. ROLLBACK командасы аяқталмаған транзакцияның барлық өзгерісін болдырмайды.

Кестелерден басқа деректер базасында қосымша нысандар болады [11]. **Индекс** – өрістің мәндері бойынша іздеуді жылдамдататын көмекші құрылым; ол кітаптың соңындағы пәндік көрсеткішке ұқсайды. **Көрініс** (VIEW) – атауы бар сақталған SELECT сұратымы; ол деректерді өзі сақтамайды, бірақ кесте сияқты қолданылады. **Триггер** – кестеде INSERT, UPDATE немесе DELETE командасы орындалғанда автоматты түрде іске қосылатын SQL командалары; оны шектеулермен жазуға болмайтын күрделі ережелерді тексеру үшін пайдаланады.

Деректер базасы мен SQL тілі отандық оқу құралдарында қазақ тілінде жан-жақты баяндалған (Г.Г. Бегаришева [1], С.К. Жұмағулова [3], М.Д. Мәнжу [4], А.К. Мұқашева және т.б. [7], А.А. Шарипбай және т.б. [9]). ДББЖ-ның құрылымы мен жұмыс істеу принциптері Х.Б. Исмаилов [5] және Р.Э. Мамедли [10] еңбектерінде қарастырылған. Курстық жұмыста осы еңбектер мен пәннің оқу-әдістемелік кешені [2] негізге алынды.

### 1.2 Деректердің негізгі модельдері

**Деректер моделі** – деректердің қалай ұйымдастырылатынын және олардың арасындағы байланыстар қалай көрсетілетінін анықтайтын тәсіл [8]. Кез келген деректер моделі үш бөліктен тұрады: деректердің құрылымы, деректерге қолданылатын амалдар және бүтіндік шектеулері [6]. Тарихи тұрғыдан ең белгілі үш модель – иерархиялық, желілік және реляциялық. Олардың бір мысалдағы көрінісі 2-суретте берілген.




2-сурет. Деректердің негізгі модельдері: а) иерархиялық, ә) желілік, б) реляциялық

**Иерархиялық модельде** деректер ағаш түрінде орналасады: әр жазбаның бір ғана «ата-анасы» болады, ал бір «ата-анада» бірнеше «ұрпақ» болуы мүмкін. Бұл модель 1960 жылдары пайда болды; оның белгілі мысалы – IBM компаниясының IMS жүйесі. Университет – кафедра – пән құрылымы ағаш түрінде жақсы көрсетіледі. Бірақ бір бұйрық бірнеше білім алушыға қатысты болса, мұндай байланысты ағашта деректерді қайталамай көрсету қиын.

**Желілік модельде** бір жазба бірнеше жазбамен байланыса алады, яғни деректер граф түрінде болады. Модельдің стандарты 1971 жылы CODASYL ұйымының ұсыныстарында бекітілді. Бұл модель «көпке-көп» байланыстарды көрсете алады, бірақ деректерді іздеу үшін бағдарлама сілтемелер бойынша бір жазбадан екіншісіне «жүріп» өтуі керек, сондықтан деректер құрылымы өзгерсе, бағдарламаны да өзгертуге тура келеді [6].

**Реляциялық модельді** 1970 жылы Э. Кодд ұсынды [12]. Бұл модельде барлық деректер кестелерде сақталады, ал кестелер кілт мәндерінің сәйкестігі арқылы байланысады; жазбалар арасында физикалық сілтемелер жоқ. Пайдаланушы деректерді қалай іздеу керегін емес, тек қандай деректер керек екенін SQL тілінде жазады. Қазіргі ДББЖ-лардың көбі реляциялық модельге негізделген.

Реляциялық модельдің математикалық негізі – **реляциялық алгебра**. Ол бір немесе екі қатынасқа қолданылып, нәтижесінде жаңа қатынас беретін амалдар жиынынан тұрады [6; 12]. Кез келген SELECT сұратымын осы амалдардың тізбегі ретінде көрсетуге болады. Негізгі амалдар және олардың SQL тіліндегі баламасы 3-кестеде берілген.

Кесте-3

Реляциялық алгебраның негізгі амалдары және олардың SQL баламасы

| Амал | Мағынасы | SQL баламасы |
|---|---|---|
| Таңдау (σ) | шартқа сәйкес кортеждерді алу | WHERE |
| Проекция (π) | керекті атрибуттарды ғана алу | SELECT өрістер тізімі |
| Біріктіру (⋈) | екі қатынасты ортақ атрибут бойынша қосу | JOIN … ON |
| Бірігу (∪) | екі қатынастың барлық кортеждері | UNION |
| Қиылысу (∩) | екі қатынаста да бар кортеждер | INTERSECT |
| Айырма (−) | біріншісінде бар, екіншісінде жоқ кортеждер | EXCEPT |
| Декарттық көбейтінді (×) | кортеждердің барлық комбинациялары | CROSS JOIN |

Реляциялық модельден кейін **объектіге бағытталған** модель пайда болды: онда деректер мен оларды өңдейтін әдістер бір объектіге біріктіріледі. Соңғы жылдары NoSQL деп аталатын модельдер (құжаттық, «кілт–мән», графтық) кеңінен қолданылады. Олар өте үлкен көлемді және құрылымы тұрақсыз деректер үшін тиімді, ал құрылымы нақты және бүтіндікті қатаң талап ететін деректер үшін реляциялық модель негізгі таңдау болып қала береді [10]. Модельдердің салыстырмасы 4-кестеде берілген.

Кесте-4

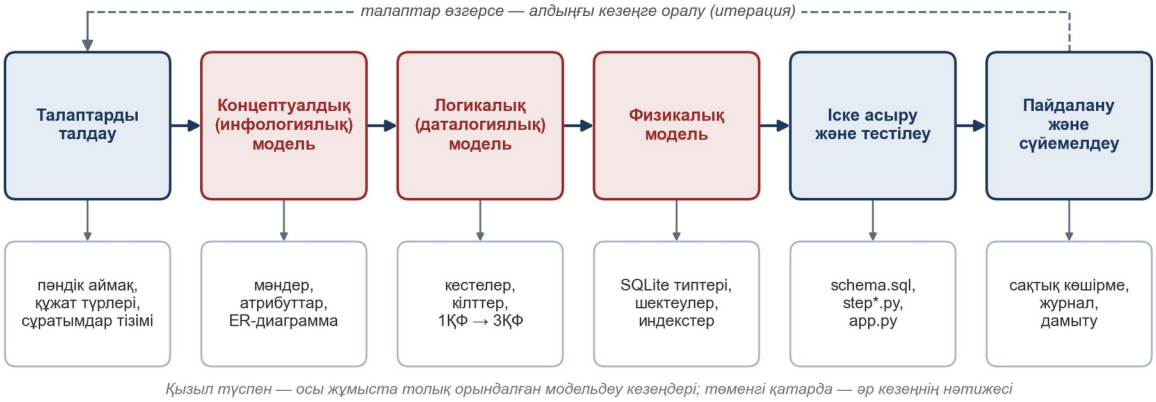
Деректер модельдерін салыстыру

| Модель | Құрылымы | Артықшылығы | Кемшілігі |
|---|---|---|---|
| Иерархиялық | ағаш | қарапайым, иерархия бойынша іздеу жылдам | «көпке-көп» байланысты көрсету қиын |
| Желілік | граф | күрделі байланыстарды көрсете алады | бағдарламалау және құрылымды өзгерту қиын |
| Реляциялық | кестелер | SQL тілі, түсінікті құрылым, деректердің тәуелсіздігі | бір объект бірнеше кестеге бөлінуі мүмкін |
| NoSQL (құжаттық) | JSON құжаттар | икемді құрылым, үлкен көлемге оңай масштабталады | бүтіндік шектеулері әлсіз, JOIN шектеулі |

Біздің пәндік аймақта бір бұйрық бірнеше білім алушыға, бір білім алушыға бірнеше құжат қатысты, ал тіркеу нөмірлері қайталанбауы керек. Сондықтан жобада реляциялық модель және SQLite ДББЖ-сы таңдалды. SQLite деректер базасы бір файлда сақталады, серверді орнатуды қажет етпейді және Python тілінде стандартты `sqlite3` модулі арқылы бірден қолданылады [13]. SQLite-та бес сақтау класы бар: NULL, INTEGER (бүтін сан), REAL (нақты сан), TEXT (мәтін) және BLOB (екілік деректер). Күн мен уақыт үшін жеке тип жоқ, сондықтан олар «ЖЖЖЖ-АА-КК» пішіміндегі мәтін ретінде сақталады [11].

### 1.3 Деректерді жобалаудың негізгі кезеңдері

Деректер базасын жобалау бірнеше кезеңнен тұрады [8]. Алдымен пәндік аймақ зерттеліп, жүйеге қойылатын талаптар жиналады. Содан кейін концептуалдық, логикалық және физикалық модельдер құрылады, соңында база жасалып, тексеріледі. Кезеңдердің реті 3-суретте көрсетілген.




3-сурет. Деректер базасын жобалау кезеңдері

**Талаптарды талдау** кезеңінде пәндік аймақтың объектілері, олардың қасиеттері, жүйені пайдаланатын адамдар және олар жиі орындайтын сұратымдар анықталады. Бұл кезеңнің нәтижесі – қандай деректер сақталатыны және олармен қандай әрекеттер жасалатыны туралы мәтіндік сипаттама. Мысалы, біздің жобада «құжатты тіркеу нөмірі бойынша табу» және «білім алушыға қатысты барлық бұйрықтарды көру» талаптары анықталды.

**Концептуалдық модель** нақты ДББЖ-ға тәуелсіз құрылады. Ол үшін П. Ченнің «мән–байланыс» (Entity–Relationship, ER) моделі жиі қолданылады [14]. **Мән** – пәндік аймақтың жеке объектісі (мысалы, Құжат, Білім алушы), **атрибут** – оның қасиеті (құжаттың күні, атауы), ал **байланыс** – мәндердің өзара қатынасы. Байланыстың үш түрі бар: «бірге-бір» (1:1), «бірге-көп» (1:N) және «көпке-көп» (M:N). Байланыстың **кардиналдығы** бір мәннің бір данасына екінші мәннің неше данасы сәйкес келетінін көрсетеді, ал **қатысу міндеттілігі** мәннің әр данасы байланысқа міндетті түрде қатыса ма, жоқ па екенін анықтайды. ER-диаграммада мәндер тіктөртбұрышпен, байланыстар ромбпен, ал атрибуттар сопақпен белгіленеді.

**Логикалық модельде** ER-модель кестелерге айналады: әр мән – кесте, әр атрибут – өріс болады. 1:1 байланыста кез келген екі кестенің біріне екіншісінің кілті қосылады. 1:N байланыс «көп» жақтағы кестеге сыртқы кілт қосу арқылы жасалады. M:N байланысты тікелей көрсету мүмкін емес, сондықтан екі кестенің кілттерін сақтайтын **аралық кесте** құрылады; оның бастапқы кілті екі сыртқы кілттен тұратын құрама кілт болады.

Кестелер **нормалданады**, яғни бір мәлімет бірнеше жерде қайталанбайтындай етіп бөлінеді. Нормалдау **функционалдық тәуелділік** ұғымына негізделеді: егер A атрибутының әр мәніне B атрибутының тек бір мәні сәйкес келсе, B атрибуты A-ға функционалды тәуелді деп аталады (A → B). Мысалы, құжат түрінің коды оның атауын анықтайды: «СИЛ» →

«Силлабус». Нормалданбаған кестеде үш түрлі **аномалия** туындайды [8; 9]: енгізу аномалиясы – бір деректі енгізу үшін басқа, әзірге жоқ деректі де енгізуге мәжбүр болу; өзгерту аномалиясы – бір мәнді өзгерткенде оны бірнеше жолда өзгерту қажеттігі; жою аномалиясы – бір жолды жойғанда басқа керекті мәліметтің де жоғалуы.

Практикада алғашқы үш қалыпты форма (ҚФ) жиі қолданылады, ал кейде Бойс–Кодд қалыпты формасы (БКҚФ) да тексеріледі. Әр келесі форма алдыңғысының талаптарын толық қамтиды. Қалыпты формалардың талаптары және олардың біздің жобада орындалуы 5-кестеде көрсетілген.

Кесте-5

Қалыпты формалардың талаптары және олардың жобада орындалуы

| Форма | Талабы | EduDocs жобасындағы мысал |
|---|---|---|
| 1ҚФ | әр өрісте бір ғана атомарлы мән, қайталанатын топтар жоқ | бұйрықтағы білім алушылар бір өріске үтірмен жазылмай, `document_students` кестесінде сақталады |
| 2ҚФ | 1ҚФ + кілт емес өрістер бүкіл құрама кілтке тәуелді | `document_students.note` (`document_id`, `student_id`) жұбына тәуелді |
| 3ҚФ | 2ҚФ + кілт емес өрістер арасында тәуелділік жоқ | құжат түрінің атауы мен сақтау мерзімі `document_types` кестесінде |
| БКҚФ | әр детерминант – кандидат кілт | `reg_number`, `record_book` – UNIQUE балама кілттер |

**Физикалық модельде** нақты ДББЖ ескеріледі: өрістердің типтері, шектеулер, индекстер, көріністер мен триггерлер анықталады. SQLite-та өрістің типі қатаң тексерілмейді, сондықтан күн сияқты мәндердің пішімін CHECK шектеуімен қосымша тексерген дұрыс [11]. Индекстер тек жиі іздейтін өрістерге құрылады, өйткені әр индекс қосымша орын алады және жазбаны енгізу мен өзгертуді сәл баяулатады.

Соңғы кезеңде схема таңдалған ДББЖ-да құрылып, тестілік деректермен толтырылады және сұратымдар арқылы тексеріледі. Тексеру барысында табылған қателер алдыңғы кезеңдерге қайта оралып түзетіледі, сондықтан жобалау үдерісі итеративті болып табылады. Осылайша, әдебиеттерге шолу нәтижесінде жобаның теориялық негізі анықталды: реляциялық модель, ER-модельдеу, үшінші қалыпты формаға дейін нормалдау және SQLite ДББЖ-сы.

## 2 Мемлекеттік жоғары оқу орнындағы оқу үдерісіне қатысты құжаттарды есепке алуға арналған деректер базасын құрудың практикалық негіздері

### 2.1 Пәндік аймақ бойынша деректер базасының жалпы сипаттамасы

**Жүйенің мақсаты** – кафедралар мен тіркеу офисі дайындайтын оқу құжаттарын бір тізілімде тіркеу. Әр құжатқа бірегей тіркеу нөмірі беріледі, оның түрі, күні, оқу жылы, жауапты бөлімшесі мен қызметкері және мәртебесі сақталады. Кейбір құжаттар нақты білім алушыларға қатысты болады: мысалы, қабылдау туралы бұйрық бірнеше білім алушыға шығарылады, ал анықтама бір білім алушыға беріледі.

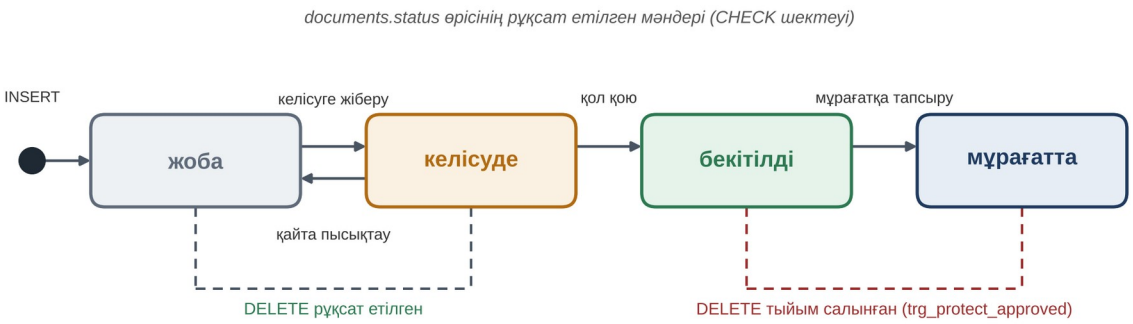
Пәндік аймақты талдау нәтижесінде төрт санатқа бөлінген сегіз құжат түрі таңдалды (6-кесте). Тіркеу нөмірі «түр коды – жыл / реттік нөмір» пішімінде жазылады, мысалы СИЛ-2026/001 – 2026 жылы тіркелген бірінші силлабус. Әр түрдің сақтау мерзімі де көрсетіледі.

Кесте-6

Құжат түрлерінің анықтамалығы (`document_types` кестесі)

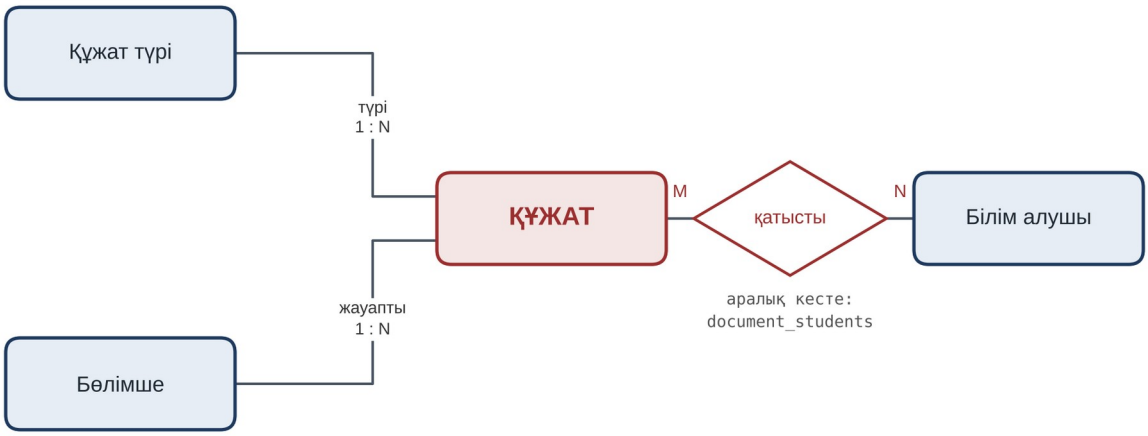
| Коды | Құжат түрі | Санаты | Сақтау мерзімі, жыл |
|---|---|---|---|
| СИЛ | Силлабус | оқу-әдістемелік | 5 |
| ОЖЖ | Оқу жұмыс жоспары | оқу-әдістемелік | 5 |
| ПОӘК | Пәннің оқу-әдістемелік кешені | оқу-әдістемелік | 5 |
| ВЕД | Емтихан ведомосі | аттестаттау | 10 |
| КЖ | Курстық жұмыс | аттестаттау | 3 |
| БК | Контингент бойынша бұйрық | контингент | 75 |
| АНҚ | Оқу орнынан анықтама | контингент | 3 |
| ХАТ | Кафедра отырысының хаттамасы | нормативтік | 10 |

Құжат дайындалғаннан бастап мұрағатқа тапсырылғанға дейін бірнеше кезеңнен өтеді. Бұл кезеңдер `status` өрісінде сақталады: **жоба** – құжат енді ғана тіркелді; **келісуде** – құжат тексеруде; **бекітілді** – құжатқа қол қойылды; **мұрағатта** – құжат мұрағатқа тапсырылды (4-сурет). Жоба мен келісудегі құжатты жоюға болады, ал бекітілген немесе мұрағаттағы құжатты жоюға триггер рұқсат бермейді.




4-сурет. Оқу құжатының өмірлік циклі

Талаптар бойынша төрт негізгі мән бөлінді: **Бөлімше**, **Құжат түрі**, **Құжат** және **Білім алушы**. Құжат түрі мен бөлімше құжатпен «бірге-көп» байланыста: бір түрге және бір бөлімшеге көп құжат тиесілі. Құжат пен білім алушы арасында «көпке-көп» байланыс бар: бір бұйрықта бірнеше білім алушы болуы мүмкін, ал бір білім алушыға бірнеше құжат қатысты болады. Концептуалдық модель 5-суретте көрсетілген.




5-сурет. Пәндік аймақтың концептуалдық (ER) моделі

ER-модель алты кестеге түрлендірілді (7-кесте). Төрт мән төрт кестеге айналды, «көпке-көп» байланыс үшін `document_students` аралық кестесі құрылды, ал веб-қосымшаға кіру үшін `users` кестесі қосылды.

Кесте-7

EduDocs деректер базасының кестелері

| № | Кесте | Мазмұны | Өріс | Жазба |
|---|---|---|---|---|
| 1 | `departments` | бөлімшелер (кафедралар, тіркеу офисі) | 3 | 4 |
| 2 | `document_types` | құжат түрлері | 5 | 8 |
| 3 | `documents` | құжаттарды тіркеу журналы (негізгі кесте) | 10 | 36 |
| 4 | `students` | білім алушылар | 6 | 21 |
| 5 | `document_students` | құжат – білім алушы байланысы (M:N) | 3 | 33 |
| 6 | `users` | қосымшаның пайдаланушылары | 5 | 2 |
| | **Барлығы** | | **32** | **104** |

Жүйенің негізгі кестесі – `documents`. Оның өрістері мен шектеулері 8-кестеде берілген. `type_id` және `department_id` өрістері – сыртқы кілттер, олар құжатты оның түрімен және бөлімшесімен байланыстырады.

Кесте-8

`documents` кестесінің құрылымы

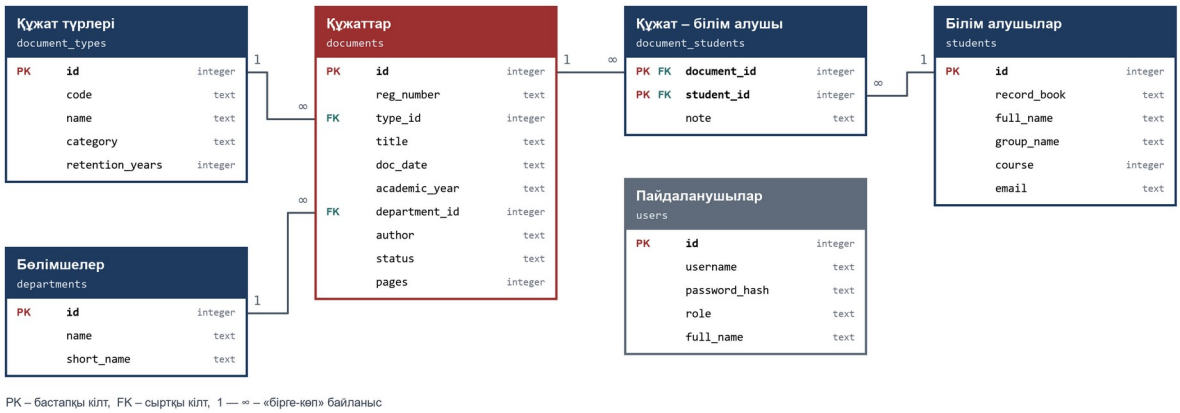
| Өріс | Типі | Шектеу | Сипаттамасы |
|---|---|---|---|
| `id` | INTEGER | PRIMARY KEY | құжаттың ішкі нөмірі |
| `reg_number` | TEXT | NOT NULL, UNIQUE | тіркеу нөмірі |
| `type_id` | INTEGER | NOT NULL, FK → document_types | құжат түрі |
| `title` | TEXT | NOT NULL | атауы |
| `doc_date` | TEXT | NOT NULL, CHECK (ЖЖЖЖ-АА-КК) | құжаттың күні |
| `academic_year` | TEXT | NOT NULL, CHECK (ЖЖЖЖ-ЖЖЖЖ) | оқу жылы |
| `department_id` | INTEGER | NOT NULL, FK → departments | жауапты бөлімше |
| `author` | TEXT | — | жауапты қызметкер |
| `status` | TEXT | NOT NULL, DEFAULT 'жоба', CHECK (4 мән) | мәртебесі |
| `pages` | INTEGER | CHECK (pages > 0) | беттер саны |

Кестелер арасында төрт сыртқы кілт бар. `documents` кестесінің екі сыртқы кілті ON DELETE RESTRICT ережесімен жарияланған: құжаттары бар бөлімшені немесе құжат түрін жоюға болмайды. `document_students` кестесінің кілттері ON DELETE CASCADE ережесімен жарияланған: құжат жойылса, оған қатысты байланыс жолдары да автоматты түрде жойылады. Аралық кестенің бастапқы кілті екі өрістен тұрады (`document_id`, `student_id`), сондықтан бір білім алушыны бір құжатқа екі рет қосуға болмайды.

Схема үшінші қалыпты формаға сәйкес келеді: құжат түрінің атауы, санаты және сақтау мерзімі әр құжатта қайталанбай, `document_types` кестесінде бір рет сақталады; бөлімшенің атауы да бөлек кестеде тұр; бұйрыққа қатысты білім алушылар бір өріске үтір арқылы жазылмай, `document_students` кестесінің жеке жолдарында сақталады.

Кестелерден басқа деректер базасында бірнеше көмекші нысан бар. Құжаттың күні мен мәртебесі бойынша іздеуді жылдамдату үшін екі **индекс** құрылды. `v_registry` **көрінісі** үш кестені біріктіріп, тіркеу журналын бір кесте сияқты береді. `trg_protect_approved` **триггері** бекітілген немесе мұрағаттағы құжатты жою әрекетін тоқтатады. Барлық CREATE командалары `schema.sql` файлында жинақталған (А қосымшасы). Деректер базасының диаграммасы 6-суретте көрсетілген; оны `step12_er_diagram.py` скрипті базаның жүйелік каталогын оқып, автоматты түрде салады.

```
CREATE TRIGGER trg_protect_approved
BEFORE DELETE ON documents
WHEN OLD.status IN ('бекітілді', 'мұрағатта')
BEGIN
    SELECT RAISE(ABORT, 'Бекітілген немесе мұрағаттағы құжатты жоюға болмайды');
END;
```




6-сурет. EduDocs деректер базасының диаграммасы

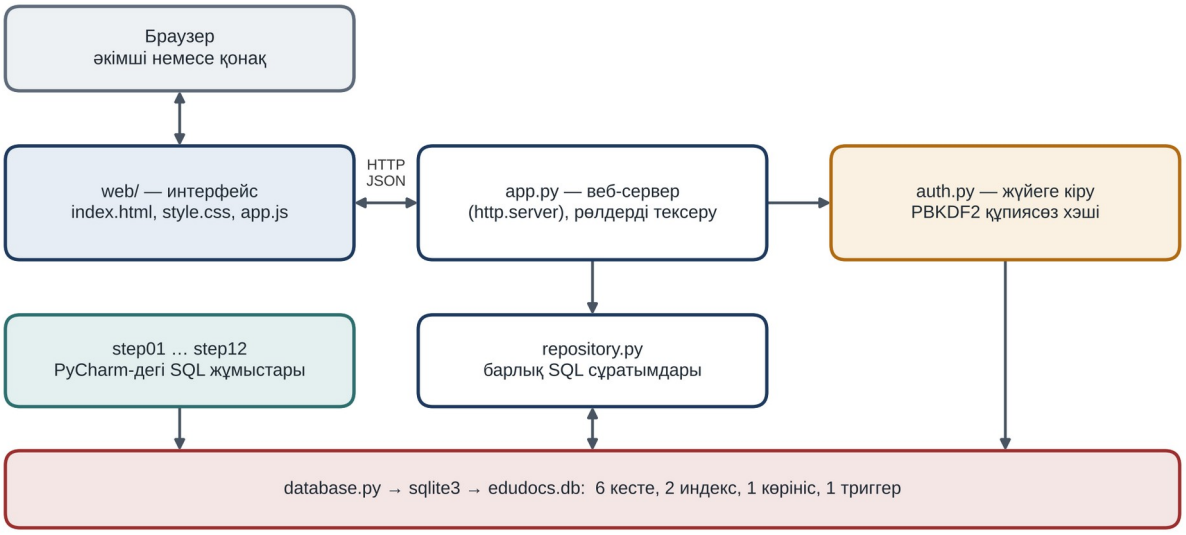
Деректер базасы демонстрациялық деректермен толтырылды: 4 бөлімше, 8 құжат түрі, 2025–2026 және 2026–2027 оқу жылдарына қатысты 36 құжат, 4 топтағы 21 білім алушы және құжат пен білім алушы арасындағы 33 байланыс. Барлығы 104 жазба. Тұлғалардың аты-жөндері ойдан құрастырылды.

### 2.2 Қосымша программасы және онымен жұмыс

Деректер базасымен жұмыс істеу үшін EduDocs атты веб-қосымша жасалды. Ол Python тілінің стандартты кітапханасымен жазылған, сондықтан қосымша пакет орнату қажет емес. PyCharm ортасында `app.py` файлы іске қосылғанда компьютердің өзінде (`127.0.0.1` адресінде) шағын веб-сервер жұмыс істей бастайды және браузер өзі ашылады [15].

Қосымшаның құрылымы 7-суретте көрсетілген. Браузердегі бет (`web` бумасындағы HTML, CSS және JavaScript файлдары) серверге сұраныс жібереді. `app.py` пайдаланушының рөлін тексеріп, сұранысты `repository.py` модуліне береді. Бұл модульде қосымшаның барлық SQL сұратымдары

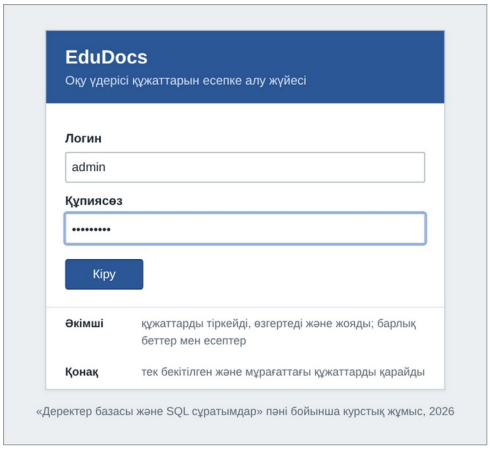
жинақталған. Нәтиже браузерге JSON пішімінде қайтарылып, кесте түрінде көрсетіледі.



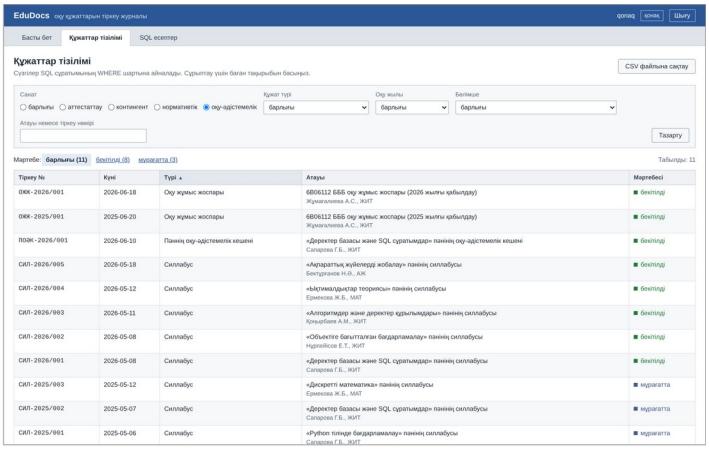

7-сурет. EduDocs қосымшасының құрылымы

Барлық сұратымдар **параметрленген** түрде жазылған: пайдаланушы енгізген мән SQL мәтініне тікелей қосылмай, «?» немесе «:атау» белгісі арқылы беріледі. Бұл SQL-инъекциядан қорғайды [13].

Қосымшада екі рөл бар: **әкімші** және **қонақ**. Қосымша ашылғанда алдымен жүйеге кіру беті шығады (8-сурет, а). Логин мен құпиясөз `users` кестесімен салыстырылады. Құпиясөз базада ашық сақталмайды: `hashlib` модулі оның хэшін (PBKDF2 алгоритмі) есептейді және базаға тек хэш жазылады. Әкімші төрт бетті көреді және құжаттарды енгізе, өзгерте, жоя алады. Қонақ үш бетті ғана көреді және оған тек «бекітілді» мен «мұрағатта» мәртебесіндегі құжаттар шығады (8-сурет, ә). Қонақтың құқығы серверде де тексеріледі: ол әкімшіге арналған әрекетті орындамақ болса, сервер қате хабарын қайтарады.




а)




ә)

8-сурет. Жүйеге кіру беті (а) және қонақ рөліндегі құжаттар тізілімі (ә)

Қосымшаны тексеру үшін `users` кестесіне екі тіркелгі енгізілген (9-кесте). Қосымшаны іске қосу үшін PyCharm ортасында `app.py` файлы

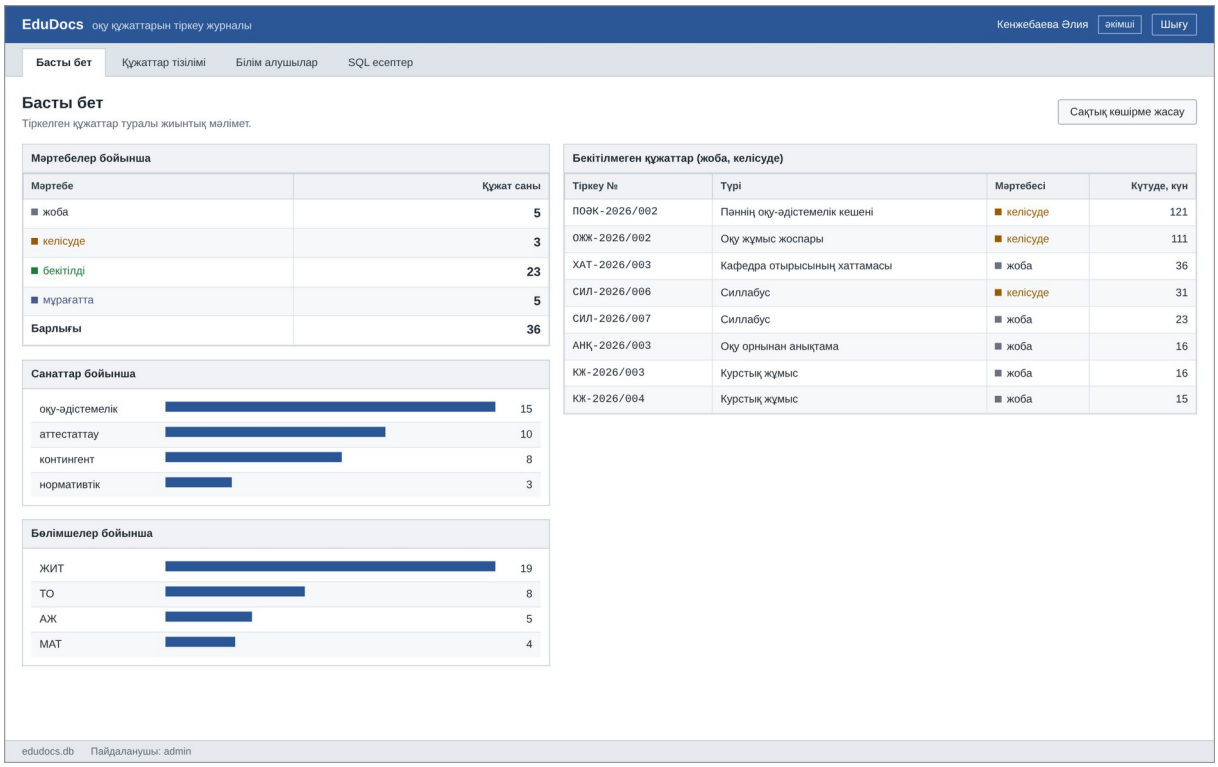
орындалады; браузер `http://127.0.0.1:8770/` адресін өзі ашады, ашылмаса бұл адресті браузерге қолмен енгізуге болады. Жұмыс аяқталғанда сервер PyCharm-дағы Stop батырмасымен тоқтатылады.

Кесте-9

Қосымшаға кіруге арналған тіркелгілер (`users` кестесі)

| Рөлі | Логин | Құпиясөз | Қолжетімді беттер |
|---|---|---|---|
| Әкімші | `admin` | `Admin2026` | Басты бет, Құжаттар тізілімі, Білім алушылар, SQL есептер (4 бет); құжаттарды енгізу, өзгерту, жою |
| Қонақ | `qonaq` | `Qonaq2026` | Басты бет, Құжаттар тізілімі, SQL есептер (3 бет); тек бекітілген және мұрағаттағы құжаттарды қарау |

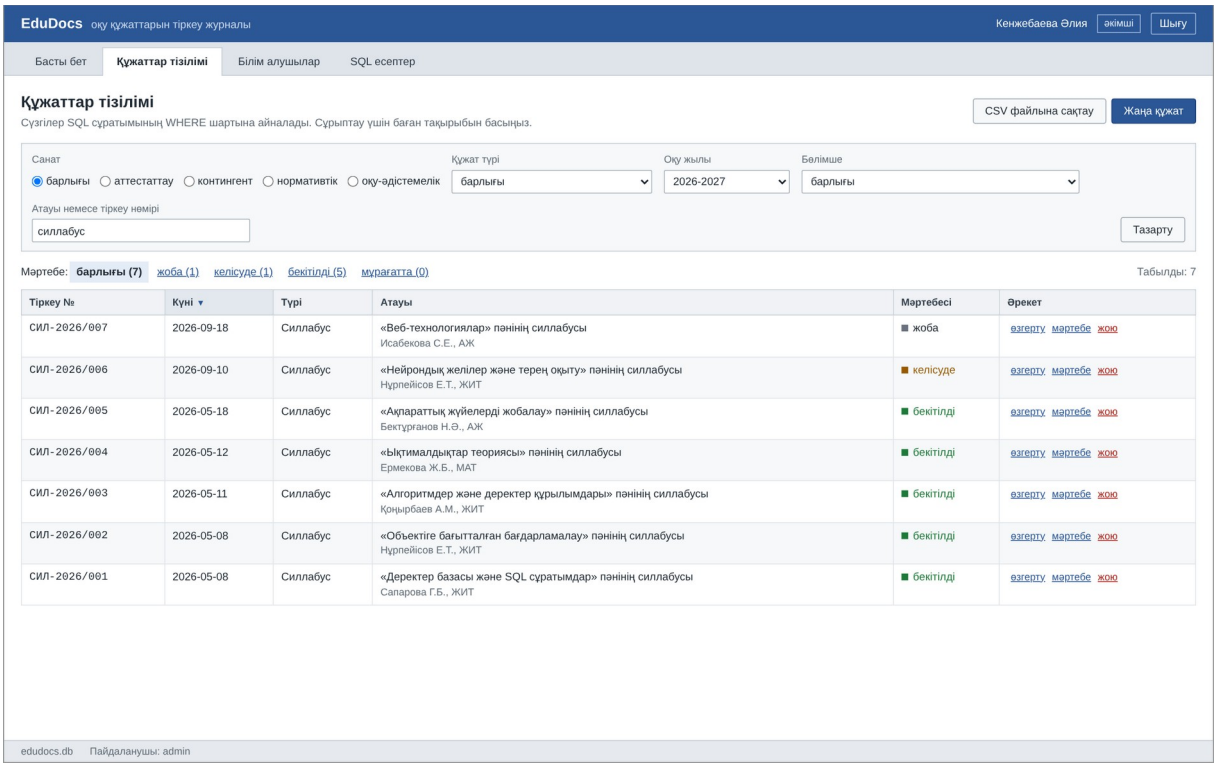
Жүйеге кіргеннен кейін «Басты бет» ашылады (9-сурет). Беттер жоғарғы жолақтағы қойындылар арқылы ауыстырылады. Басты бетте құжаттардың мәртебелер бойынша саны, санаттар мен бөлімшелер бойынша диаграммалар және әлі бекітілмеген құжаттар тізімі (әр құжаттың қанша күн күтіп тұрғаны) көрсетіледі. Бұл деректер COUNT және GROUP BY сұратымдарымен алынады. Әкімші осы беттегі «Сақтық көшірме жасау» батырмасы арқылы базаның көшірмесін жасай алады.




9-сурет. EduDocs қосымшасының басты беті

«Құжаттар тізілімі» бетінде `v_registry` көрінісінің деректері кесте түрінде шығады (10-сурет). Кестенің үстіндегі іздеу формасында құжаттың санатын, түрін, оқу жылын және бөлімшесін таңдауға, сондай-ақ атауы немесе тіркеу нөмірі бойынша іздеуге болады. Барлық сүзгілер SQL сұратымының WHERE шартына айналады. Форманың астындағы мәртебе сілтемелерінде әр

мәртебедегі құжаттар саны көрсетіледі. Баған тақырыбын басқанда кесте сол баған бойынша сұрыпталады: бірінші рет басқанда өсу (▲), екінші рет басқанда кему (▼) ретімен. Суретте «силлабус» сөзі мен 2026–2027 оқу жылы бойынша сүзгілеу нәтижесі (7 құжат) күні бойынша кему ретімен көрсетілген. Тізімді CSV файлына да сақтауға болады.




10-сурет. Құжаттар тізілімі: атауы мен оқу жылы бойынша сүзгілеу

«Жаңа құжат» батырмасы тіркеу формасын ашады (В қосымшасы, В.1-сурет). Құжат түрі таңдалғанда келесі тіркеу нөмірі MAX() функциясы арқылы автоматты түрде ұсынылады. Міндетті өрістер толтырылмаса немесе тіркеу нөмірі қайталанса, пайдаланушыға қате туралы хабар шығады. Тізімдегі құжат жолын басқанда құжат картасы ашылады (В.2-сурет): мұнда құжаттың толық мәліметтері және оған қатысты білім алушылар (`document_students` кестесі) көрсетіледі. Картадан білім алушыны құжатқа қосуға немесе байланысты алып тастауға болады.

«Білім алушылар» бетінде білім алушыны аты-жөні немесе ID нөмірі бойынша іздеуге және тобы бойынша сүзуге болады. Білім алушы таңдалғанда оған қатысты барлық құжаттар шығады (В.3-сурет). «SQL есептер» бетінде дайын есептер бар: тіркеу журналы, құжат түрлері бойынша статистика, бөлімшелер бойынша құжаттар саны, NOT EXISTS және LEFT JOIN мысалдары (В.4-сурет). Әкімші есептің SQL мәтінін өзгертіп, қайта орындай алады, ал қонақ тек бес ашық есепті көреді. Есептер базаға тек оқу режимінде ашылған қосылыммен орындалады, сондықтан кездейсоқ жазылған DELETE командасы деректерді өзгерте алмайды.

### 2.3 Деректер базасын PyCharm ортасында өңдеу жұмыстары

Курстық жұмыстың міндеттері PyCharm ортасында кезекпен орындалатын 12 скрипт арқылы жүзеге асырылды (10-кесте). Әр скрипт жеке іске қосылады (Run немесе Ctrl+Shift+F10) және нәтижесін Run терезесіне кесте түрінде шығарады; `run_all_steps.py` скрипті барлық қадамды қатарынан орындайды. SQL командалары `sqlite3` модулінің `connect()`, `cursor()`, `execute()`, `executemany()`, `fetchone()`, `fetchall()`, `commit()`, `rollback()` және `close()` әдістері арқылы орындалады [13]. Төмендегі суреттер PyCharm ортасынан алынды: сол жақта скрипттің коды, оң жақта Run терезесіндегі нәтиже көрсетілген. Run терезесінің соңғы жолындағы 0 шығу коды бағдарламаның қатесіз аяқталғанын білдіреді. Құрылған `edudocs.db` файлын DB Browser for SQLite [16] бағдарламасымен де ашып көруге болады.

Кесте-10

PyCharm жобасындағы скрипттер және олардың міндеттері

| Скрипт | Курстық жұмыс міндеті | SQL / Python құралдары |
|---|---|---|
| `step01_connect.py` | ДБ құру, қосу және ажырату | connect(), cursor(), PRAGMA, close(), ATTACH, DETACH |
| `step02_create_tables.py` | кестелерді құру, байланыстарды орнату | CREATE TABLE, FOREIGN KEY, ALTER TABLE, DROP TABLE |
| `step03_insert.py` | деректерді енгізу | INSERT, executemany(), INSERT … SELECT, commit() |
| `step04_update_delete.py` | деректерді өзгерту және жою | UPDATE, DELETE, CASCADE, RESTRICT |
| `step05_select_simple.py` | бір кестеден іріктеу | SELECT, AS, DISTINCT, LIMIT, fetchone() |
| `step06_filter.py` | деректерді сүзгілеу | WHERE, AND, OR, LIKE, BETWEEN, IN, IS NULL |
| `step07_group_sort.py` | жолдарды топтау және сұрыптау | ORDER BY, GROUP BY, HAVING, COUNT, SUM, AVG |
| `step08_join.py` | бірнеше кестеден іріктеу | INNER JOIN, LEFT JOIN |
| `step09_subquery_union.py` | қосымша: кірістірілген сұратымдар | SELECT ішіндегі SELECT, IN, NOT EXISTS, UNION |
| `step10_view_trigger.py` | қосымша: көрініс және триггер | CREATE VIEW, CREATE TRIGGER |
| `step11_transaction_index.py` | қосымша: транзакция және индекс | BEGIN, COMMIT, ROLLBACK, CREATE INDEX, EXPLAIN |
| `step12_er_diagram.py` | ДБ диаграммасын құру | PRAGMA table_info, PRAGMA foreign_key_list |

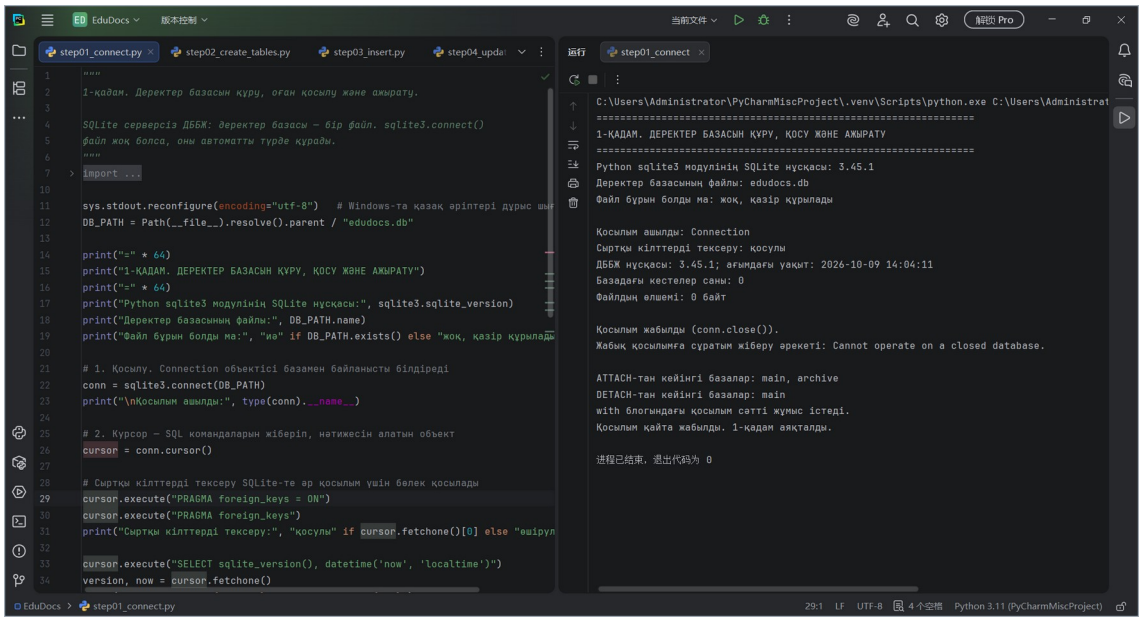
#### 2.3.1 Деректер базасын құру, қосу және ажырату

SQLite – серверсіз ДББЖ, деректер базасы бір файлда сақталады. `sqlite3.connect()` функциясы файл жоқ болса, оны жаңадан құрады және

Connection объектісін қайтарады. SQL командалары Cursor объектісі арқылы жіберіледі. Сыртқы кілттерді тексеру SQLite-та әр қосылым үшін `PRAGMA foreign_keys = ON` командасымен бөлек қосылады. Жұмыс аяқталғанда қосылым `close()` әдісімен жабылады (ажыратылады); жабық қосылымға сұратым жіберу қатеге әкеледі. Бір қосылымға екінші базаны да қосуға болады: `ATTACH DATABASE` командасы `archive.db` мұрағат файлын қосады, ал `DETACH DATABASE` оны ажыратады (11-сурет).

```
conn = sqlite3.connect(DB_PATH)
cursor = conn.cursor()
cursor.execute("PRAGMA foreign_keys = ON")
conn.close()

conn = sqlite3.connect(DB_PATH)
conn.execute("ATTACH DATABASE ? AS archive", (str(ARCHIVE_PATH),))
conn.execute("DETACH DATABASE archive")
```

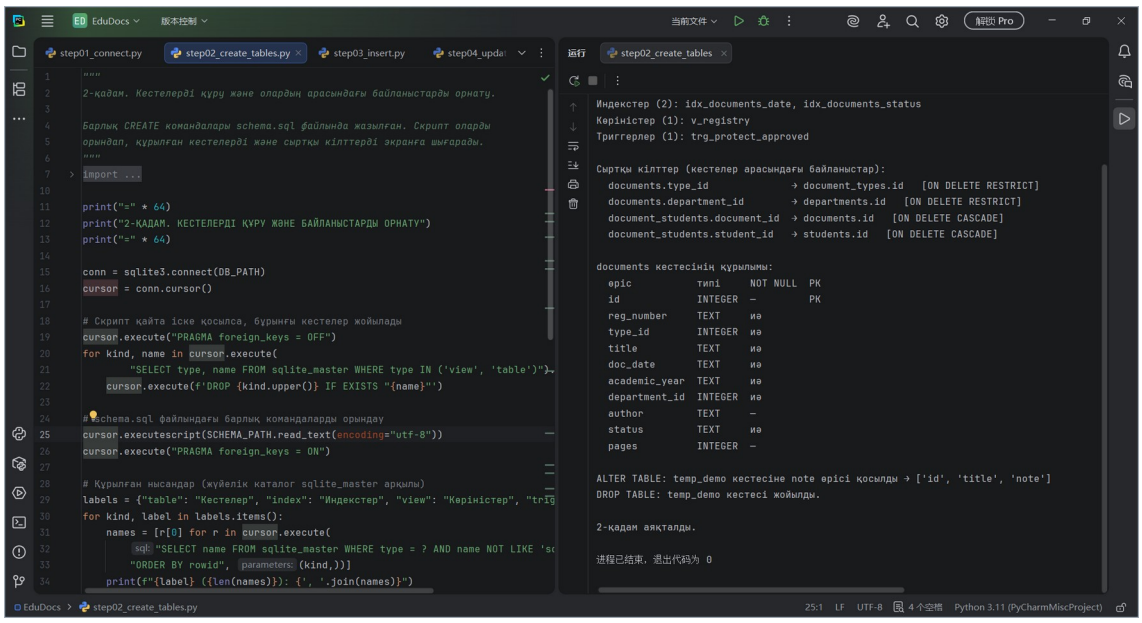



11-сурет. PyCharm ортасы: step01_connect.py – деректер базасын құру, қосу және ажырату

### 2.3.2 Кестелерді құру және байланыстарды орнату

Кестелер `schema.sql` файлындағы CREATE TABLE командаларымен құрылады; скрипт бұл файлды `executescript()` әдісімен орындайды. Құрылғаннан кейін жүйелік `sqlite_master` кестесінен барлық нысандар (6 кесте, 2 индекс, 1 көрініс, 1 триггер), ал `PRAGMA foreign_key_list` командасы арқылы төрт сыртқы кілт және олардың RESTRICT, CASCADE ережелері оқылады. Сонымен қатар ALTER TABLE (өріс қосу) және DROP TABLE (кестені жою) командалары уақытша кестеде көрсетілген (12-сурет).

```
CREATE TABLE document_students (
    document_id  INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
    student_id   INTEGER NOT NULL REFERENCES students(id)  ON DELETE CASCADE,
    note         TEXT,                                  -- ескерту
    PRIMARY KEY (document_id, student_id)
);
```

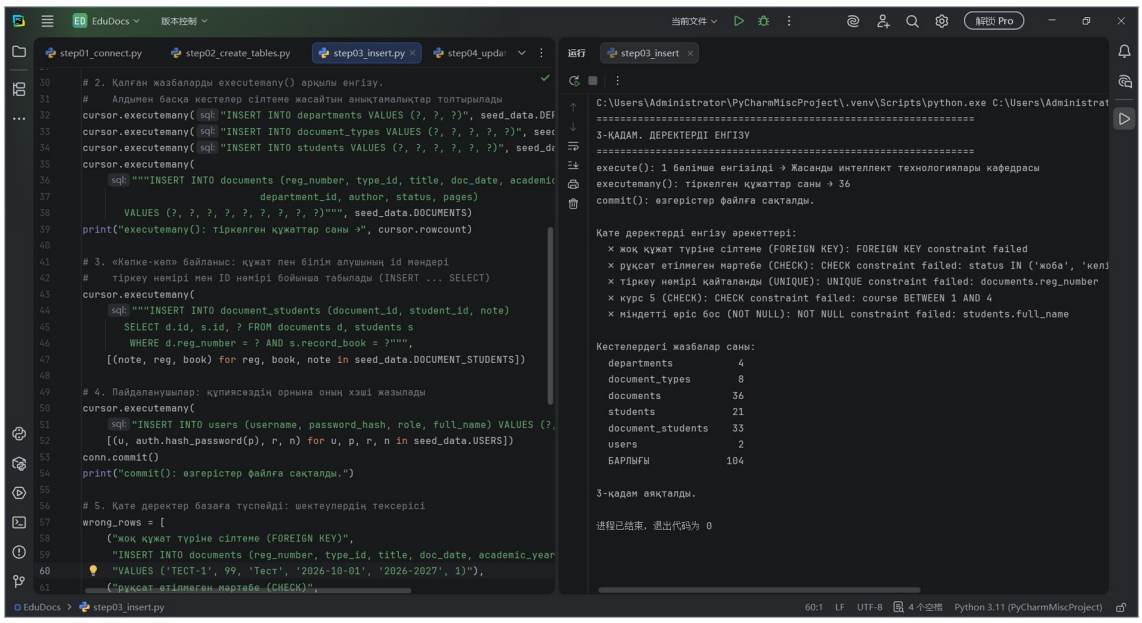



12-сурет. PyCharm ортасы: step02_create_tables.py – кестелер мен байланыстарды құру

### 2.3.3 Деректерді енгізу, өзгерту және жою

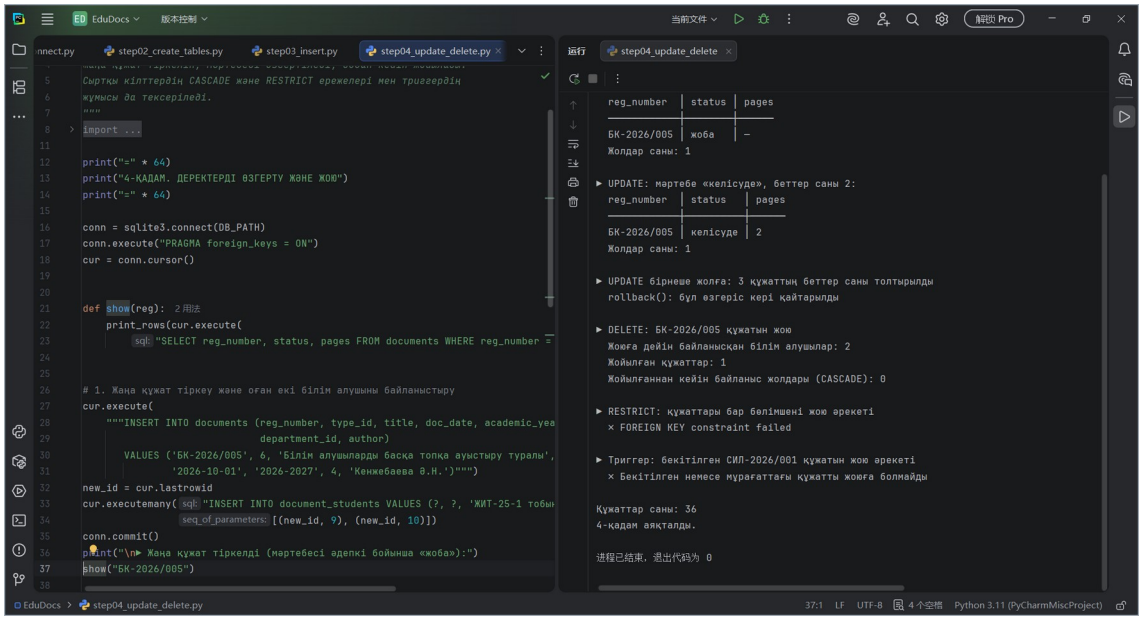
Бір жазба `execute()`, ал көп жазба `executemany()` әдісімен енгізіледі. Мәндер SQL мәтініне тікелей жазылмай, «?» параметрлері арқылы беріледі. «Көпке-көп» кестесіне құжат пен білім алушының id мәндері INSERT … SELECT құрылымы арқылы табылады. Енгізу `commit()` әдісімен сақталады. Скрипт шектеулердің жұмысын да тексереді: жоқ құжат түріне сілтеме (FOREIGN KEY), рұқсат етілмеген мәртебе (CHECK), қайталанған тіркеу нөмірі (UNIQUE), 5-курс (CHECK) және бос аты-жөні (NOT NULL) базаға түспейді. Нәтижесінде 6 кестеде 104 жазба сақталды (13-сурет).

```
cursor.executemany("INSERT INTO departments VALUES (?, ?, ?)",
seed_data.DEPARTMENTS[1:])
cursor.executemany("INSERT INTO document_types VALUES (?, ?, ?, ?, ?)",
seed_data.DOCUMENT_TYPES)
cursor.executemany("INSERT INTO students VALUES (?, ?, ?, ?, ?, ?)",
seed_data.STUDENTS)
cursor.executemany(
    """INSERT INTO documents (reg_number, type_id, title, doc_date, academic_year,
                              department_id, author, status, pages)
       VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)""", seed_data.DOCUMENTS)
```



13-сурет. PyCharm ортасы: step03_insert.py – деректерді енгізу және шектеулерді тексеру

Төртінші скриптте жаңа бұйрық тіркеліп, оған екі білім алушы қосылады, содан кейін UPDATE командасы оның мәртебесін «келісуде» етіп өзгертеді. DELETE командасы бұйрықты жойғанда, ON DELETE CASCADE ережесі бойынша оған қатысты екі байланыс жолы да жойылады. Құжаттары бар бөлімшені жою әрекетін RESTRICT ережесі, ал бекітілген силлабусты жою әрекетін `trg_protect_approved` триггері тоқтатады (14-сурет). Толық мәтіні Б қосымшасында берілген.

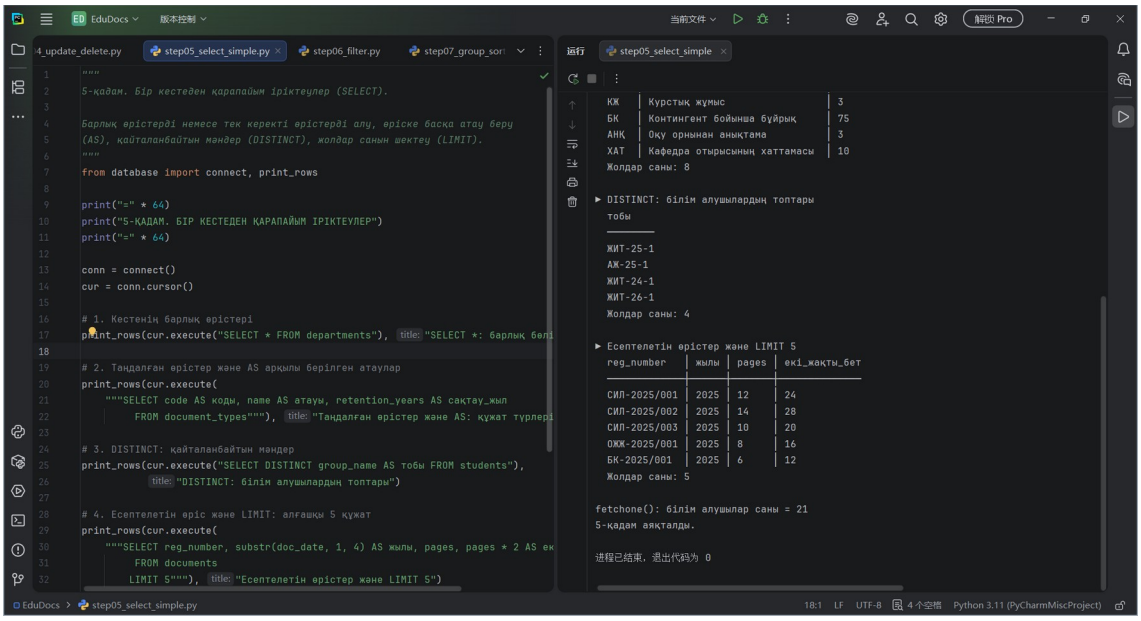


14-сурет. PyCharm ортасы: step04_update_delete.py – деректерді өзгерту және жою

### 2.3.4 Бір кестеден қарапайым іріктеулер

SELECT командасы бір кестеден барлық өрістерді (*) немесе тек керекті өрістерді алады. AS сөзі өріске жаңа атау береді, DISTINCT қайталанатын

мәндерді алып тастайды, LIMIT шығатын жолдар санын шектейді. Сұратымда есептелетін өріс те болуы мүмкін, мысалы `substr(doc_date, 1, 4)` құжаттың жылын бөліп алады. Бір мәнді `fetchone()`, ал барлық жолдарды `fetchall()` әдісі қайтарады (15-сурет).

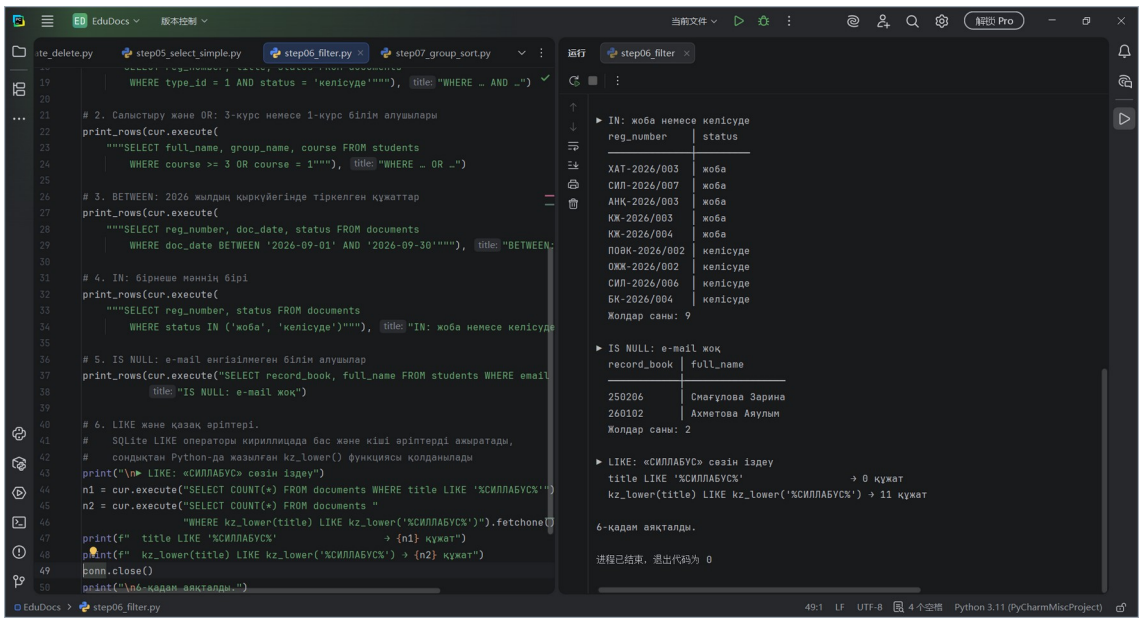



15-сурет. PyCharm ортасы: step05_select_simple.py – бір кестеден іріктеу

### 2.3.5 Деректерді сүзгілеу

WHERE сөйлемі шартқа сәйкес жолдарды ғана таңдайды. Скриптте салыстыру операторлары (=, >=), логикалық AND және OR, BETWEEN (күндер аралығы), IN (бірнеше мәннің бірі) және IS NULL (бос мән) қолданылды. Сүзгілеу кезінде бір ерекшелік анықталды: SQLite-тың LIKE операторы кириллица әріптерінде бас және кіші әріпті ажыратады, сондықтан «СИЛЛАБУС» сөзі бойынша іздеу 0 құжат табады. Python-да жазылған `kz_lower()` функциясы `create_function()` әдісімен базаға тіркелгеннен кейін, сол іздеу 11 құжатты тапты (16-сурет).

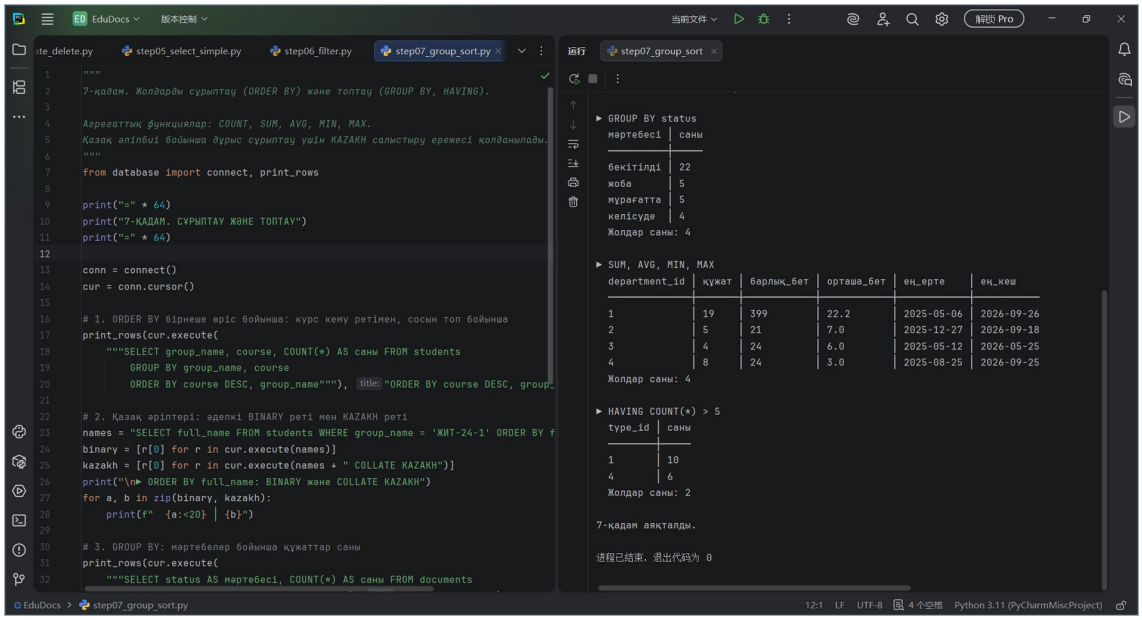
```
n1 = cur.execute("SELECT COUNT(*) FROM documents WHERE title LIKE '%СИЛЛАБУС
%'").fetchone()[0]
n2 = cur.execute("SELECT COUNT(*) FROM documents "
                 "WHERE kz_lower(title) LIKE kz_lower('%СИЛЛАБУС%')").fetchone()[0]
```



16-сурет. PyCharm ортасы: step06_filter.py – деректерді сүзгілеу

### 2.3.6 Жолдарды топтау және сұрыптау

ORDER BY сөйлемі жолдарды бір немесе бірнеше өріс бойынша өсу (ASC) немесе кему (DESC) ретімен сұрыптайды. GROUP BY жолдарды топтарға бөледі, ал COUNT, SUM, AVG, MIN, MAX агрегаттық функциялары әр топ бойынша мән есептейді. HAVING сөйлемі топтарды сүзеді, мысалы 5-тен көп құжаты бар түрлерді таңдайды. Әдепкі сұрыптау Ә, Қ, Ө, І сияқты қазақ әріптерін тізімнің соңына қояды, сондықтан `create_collation()` әдісімен KAZAKH салыстыру ережесі тіркелді; ол аты-жөндерді қазақ әліпбиі бойынша дұрыс сұрыптайды (17-сурет).

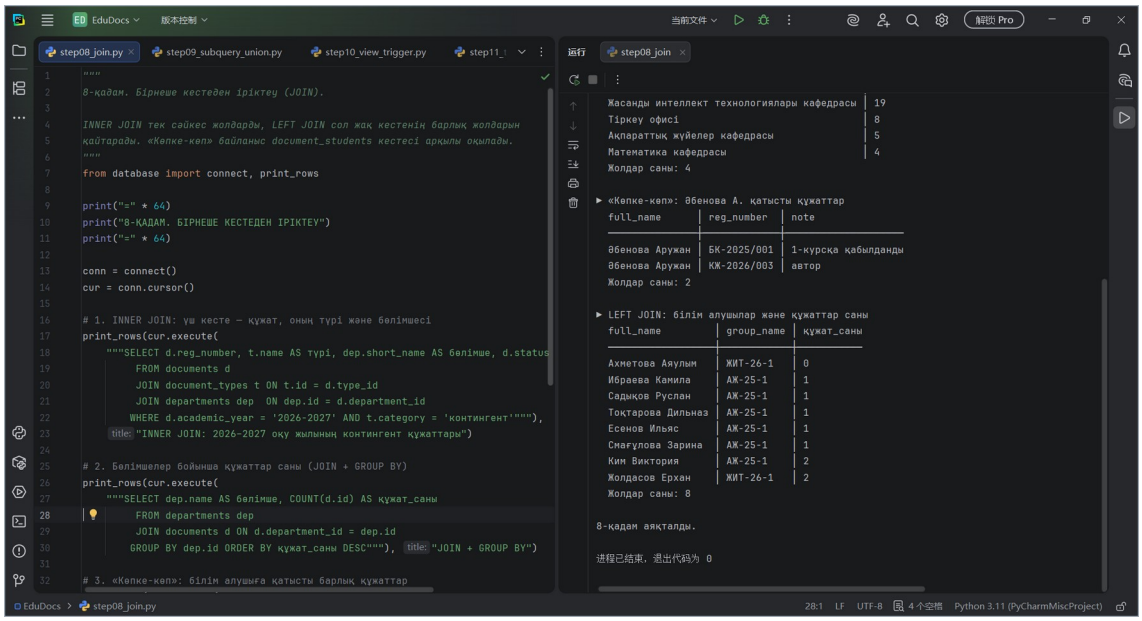


17-сурет. PyCharm ортасы: step07_group_sort.py – топтау және сұрыптау

### 2.3.7 Бірнеше кестеден іріктеу

Бірнеше кестедегі деректер JOIN арқылы біріктіріледі. INNER JOIN екі кестеде де сәйкес жолдары бар жазбаларды ғана қайтарады: мысалы, құжатты оның түрімен және бөлімшесімен біріктіру. «Көпке-көп» байланыс екі JOIN арқылы оқылады: білім алушы → `document_students` → құжат. LEFT JOIN сол жақ кестенің барлық жолдарын қайтарады, сәйкестік жоқ болса NULL береді; сондықтан бірде-бір құжаты жоқ білім алушы да 0 санымен шығады (18-сурет).

```
    """SELECT s.full_name, d.reg_number, ds.note
         FROM students s
         JOIN document_students ds ON ds.student_id = s.id
         JOIN documents d          ON d.id = ds.document_id
        WHERE s.record_book = '250101'"""), "«Көпке-көп»: Әбенова А. қатысты
құжаттар")
```




18-сурет. PyCharm ортасы: step08_join.py – бірнеше кестеден іріктеу

### 2.3.8 Қосымша SQL мүмкіндіктері

Курстық жұмыстың негізгі міндеттерінен бөлек бірнеше қосымша мүмкіндік қарастырылды. Кірістірілген сұратымдар (subquery) беттер саны орташадан көп құжаттарды, бұйрықтарда аты бар білім алушыларды (IN) және 2026–2027 оқу жылында әлі бірде-бір құжаты жоқ түрді (NOT EXISTS) тапты; UNION екі сұратымның нәтижесін біріктірді (В.5-сурет). Көрініс `v_registry` үш кестені JOIN жазбай-ақ кесте сияқты оқуға мүмкіндік берді, ал триггер бекітілген ведомосты жою әрекетін тоқтатты (В.6-сурет). Транзакция мысалында екі кестедегі өзгеріс COMMIT арқылы бірге сақталды, ал қатесі бар бұйрық ROLLBACK арқылы толығымен болдырылмады; индекстің әсері EXPLAIN QUERY PLAN командасымен тексерілді (В.7-сурет). Соңғы скрипт деректер базасының диаграммасын салды (В.8-сурет).

## 3 Оқу үдерісі құжаттарын есепке алуға арналған деректер базасын зерттеу нәтижелері және қорытынды

Жұмыс нәтижесінде курстық жұмыстың барлық міндеттері орындалды (11-кесте).

Кесте-11

Курстық жұмыс міндеттерінің орындалуы

| Міндет | Орындалу нәтижесі | Көрсетілген орны |
|---|---|---|
| ДБ құру, қосу және ажырату | edudocs.db файлы; connect(), close(), ATTACH, DETACH | 2.3.1, 11-сурет |
| Кестелерді құру | 6 кесте; NOT NULL, UNIQUE, CHECK шектеулері | 2.1, 2.3.2, А қосымшасы |
| Кестелер арасында байланыс орнату | 4 сыртқы кілт; CASCADE және RESTRICT ережелері | 2.1, 12-сурет |
| ДБ диаграммасын құру | жүйелік каталогтан автоматты салынған диаграмма | 6-сурет |
| Деректерді енгізу, өзгерту, жою | 104 жазба; INSERT, UPDATE, DELETE | 13, 14-суреттер |
| Бір кестеден іріктеу | SELECT, AS, DISTINCT, LIMIT | 15-сурет |
| Деректерді сүзгілеу | WHERE, LIKE, BETWEEN, IN, IS NULL | 16-сурет |
| Жолдарды топтау және сұрыптау | ORDER BY, GROUP BY, HAVING, KAZAKH | 17-сурет |
| Бірнеше кестеден іріктеу | INNER JOIN, LEFT JOIN, «көпке-көп» байланыс | 18-сурет |
| Қосымша (интерфейс) | веб-қосымша: кіру (9-кесте), екі рөл, сүзгілеу, сұрыптау | 2.2, 8–10-суреттер, В.1–В.4 |

Деректер базасының соңғы күйінде 6 кестеде 104 жазба сақталады. Тіркелген 36 құжаттың 23-і бекітілген, 5-еуі мұрағатта, 3-еуі келісуде және 5-еуі жоба күйінде. Құжаттардың 15-і оқу-әдістемелік, 10-ы аттестаттау, 8-і контингент және 3-еуі нормативтік санатқа жатады. Ең көп құжатты Жасанды интеллект технологиялары кафедрасы (19 құжат) тіркеген.

Деректердің бүтіндігі де тексерілді. Қате деректерді енгізудің бес әрекеті (жоқ құжат түріне сілтеме, рұқсат етілмеген мәртебе, қайталанған тіркеу нөмірі, 5-курс және бос аты-жөні) шектеулермен тоқтатылды. Құжаттары бар бөлімшені жою әрекетін RESTRICT ережесі, ал бекітілген құжатты жою әрекетін триггер болдырмады. Транзакция мысалында қатесі бар бұйрық ROLLBACK арқылы толық болдырылмады: құжаттар саны транзакцияға дейін де, кейін де 36 болып қалды.

Веб-қосымша екі рөлмен тексерілді. Қонақ ретінде кіргенде тізілімде тек «бекітілді» және «мұрағатта» мәртебесіндегі 28 құжат көрсетіледі, ал құжатты өзгерту немесе жою сұрауына сервер 403 қатесін қайтарады. Әкімші ретінде жаңа құжатты тіркеу, мәртебесін өзгерту, жою және білім алушыны құжатқа

қосу әрекеттері дұрыс орындалды; қайталанған тіркеу нөмірін енгізу әрекетіне UNIQUE шектеуі қате хабарын берді.

Индекстердің жұмысы EXPLAIN QUERY PLAN командасымен тексерілді: күн бойынша іздеу сұратымы «SEARCH documents USING INDEX idx_documents_date» жоспарымен, яғни бүкіл кестені қарап шықпай, индекс арқылы орындалады [11].

Жұмыстың тағы бір нәтижесі – қазақ тіліндегі деректермен жұмыстың екі мәселесі анықталып, шешілді. Біріншіден, LIKE операторы кириллицада бас және кіші әріпті ажыратады; бұл `kz_lower()` функциясымен шешілді. Екіншіден, әдепкі сұрыптау қазақ әріптерін әліпби ретімен орналастырмайды; бұл KAZAKH салыстыру ережесімен шешілді [11].

Алынған нәтижелерді қазір қолданылып жүрген жүйелермен салыстыруға болады. Қазақстан ЖОО-ларында оқу үдерісі Platonus, Univer сияқты автоматтандырылған ақпараттық жүйелерде, ал ұйымның құжат айналымы электрондық құжат айналымы жүйелерінде жүргізіледі. Олар серверлік ДББЖ-ға негізделген, көп пайдаланушымен қатар жұмыс істейді және электрондық цифрлық қолтаңбаны қолдайды. Біздің жоба мұндай жүйелерді алмастыруды көздемейді: ол бір кафедра немесе тіркеу офисі деңгейіндегі тіркеу журналын шағын көлемде модельдейді. Дегенмен негізгі тәсілдер ортақ: құжаттың бірегей тіркеу нөмірі, мәртебелер арқылы бақыланатын өмірлік цикл, рөлдер бойынша қолжетімділік және анықтамалықтарға бөлінген нормалданған схема.

Жұмыстың **шектеулері**: SQLite бір мезгілде көп пайдаланушы жазатын жүйеге арналмаған; веб-қосымша тек бір компьютерде жұмыс істейді; құжаттың өзі (PDF файлы) базада сақталмайды. **Даму бағыттары**: PostgreSQL сияқты серверлік ДББЖ-ға көшу, құжат файлдарын сақтау, қызметкерлер үшін бөлек кесте қосу және электрондық цифрлық қолтаңбамен байланыстыру.

# ҚОРЫТЫНДЫ

Курстық жұмыста мемлекеттік жоғары оқу орнындағы оқу үдерісіне қатысты құжаттарды есепке алуға арналған реляциялық деректер базасы жобаланып, Python және SQLite құралдарымен PyCharm ортасында жасалды.

Әдебиеттерге шолу барысында деректер базасының негізгі ұғымдары, иерархиялық, желілік және реляциялық модельдер, сондай-ақ жобалау кезеңдері қарастырылды. Пәндік аймақта «көпке-көп» байланыстар бар болғандықтан және деректердің дұрыстығы маңызды болғандықтан, реляциялық модель мен SQLite ДББЖ-сы таңдалды.

Пәндік аймақты талдау нәтижесінде 8 құжат түрі және құжаттың төрт мәртебелі өмірлік циклі анықталды. ER-модель үшінші қалыпты формадағы 6 кестеге түрлендірілді; 4 сыртқы кілт, NOT NULL, UNIQUE, CHECK шектеулері, 2 индекс, 1 көрініс және 1 триггер құрылды.

PyCharm ортасында орындалған 12 скрипт курстық жұмыстың барлық міндеттерін қамтыды: деректер базасын құру, қосу және ажырату, кестелер мен байланыстарды құру, деректерді енгізу, өзгерту және жою, бір кестелі және бірнеше кестелік іріктеулер, сүзгілеу, топтау мен сұрыптау, диаграмма салу. Әзірленген EduDocs веб-қосымшасы құжаттарды тіркеуге, іздеуге, сүзгілеуге және сұрыптауға мүмкіндік береді; оған логин мен құпиясөз арқылы кіріледі, әкімші деректерді басқарады, ал қонақ тек бекітілген құжаттарды көреді.

Жұмыс барысында «Деректер базасы және SQL сұратымдар» пәні бойынша алған білім бекітіліп, нақты тақырыпқа деректер базасын жобалау мен SQL сұратымдарын жазудың практикалық дағдылары меңгерілді.

## ПАЙДАЛАНЫЛҒАН ДЕРЕК КӨЗДЕРІНІҢ ТІЗІМІ


1. Бегаришева Г.Г. Ақпараттық жүйелердегі мәліметтер базасы: оқу құралы. – Алматы: Монте Кристо, 2018. – 106 б.

2. Елибаева Г.К. «Деректер базасы және SQL сұратымдар» пәнінің оқу-әдістемелік кешені (DBSQLS 2214). – Астана: Л.Н. Гумилев атындағы ЕҰУ, 2026. – 114 б.

3. Жұмағулова С.К. Ақпараттық жүйелердегі мәліметтер базасы: оқу құралы. – Алматы: Эверо, 2020. – 100 б.

4. Мәнжу М.Д. Ақпараттық жүйелердегі мәліметтер базасы: оқу құралы. – Алматы: Экономика, 2016. – 129 б.

5. Исмаилов Х.Б. Деректер қорын басқару жүйесі. – Алматы: Эверо, 2022. – 232 б.

6. Дейт К.Дж. Введение в системы баз данных. – 8-е изд. – М.: Вильямс, 2005. – 1328 с.

7. Мұқашева А.К., Мұхамеджанова Д.М., Сүлейменова М.У. Ақпараттық жүйелердегі деректерді басқару: оқу құралы. – Алматы: TechSmith, 2023. – 116 б.

8. Коннолли Т., Бегг К. Базы данных. Проектирование, реализация и сопровождение. Теория и практика. – 3-е изд. – М.: Вильямс, 2003. – 1436 с.

9. Шарипбай А.А., Омарбекова А.С., Ниязова Р.С. Реляциялық деректер базасын құру және оны әкімшіліктеу: оқу құралы. – Алматы: Эверо, 2017. – 150 б.

10. Мамедли Р.Э. Системы управления базами данных: учебник. – СПб.: Лань, 2024. – 228 с.

11. SQLite Documentation [Электрондық ресурс]. – URL: https://www.sqlite.org/docs.html (қаралған күні: 26.09.2026).

12. Codd E.F. A Relational Model of Data for Large Shared Data Banks // Communications of the ACM. – 1970. – Vol. 13, № 6. – P. 377–387.

13. sqlite3 – DB-API 2.0 interface for SQLite databases. Python 3 Documentation [Электрондық ресурс]. – URL: https://docs.python.org/3/library/sqlite3.html (қаралған күні: 26.09.2026).

14. Chen P.P. The Entity-Relationship Model – Toward a Unified View of Data // ACM Transactions on Database Systems. – 1976. – Vol. 1, № 1. – P. 9–36.

15. http.server – HTTP servers. Python 3 Documentation [Электрондық ресурс]. – URL: https://docs.python.org/3/library/http.server.html (қаралған күні: 26.09.2026).

16. DB Browser for SQLite [Электрондық ресурс]. – URL: https://sqlitebrowser.org/ (қаралған күні: 26.09.2026).

# ҚОСЫМША А
# Деректер базасының схемасы (schema.sql)

```
-- ======================================================================
--  EduDocs — мемлекеттік ЖОО-дағы оқу үдерісіне қатысты құжаттарды
--  есепке алуға арналған деректер базасы
--  ДББЖ: SQLite 3.   Кестелер саны: 6
-- ======================================================================

-- 1. Бөлімшелер: кафедралар және тіркеу офисі
CREATE TABLE departments (
    id          INTEGER PRIMARY KEY,
    name        TEXT    NOT NULL UNIQUE,          -- толық атауы
    short_name  TEXT    NOT NULL UNIQUE           -- қысқаша атауы (ЖИТ, АЖ …)
);

-- 2. Құжат түрлері (анықтамалық)
CREATE TABLE document_types (
    id               INTEGER PRIMARY KEY,
    code             TEXT    NOT NULL UNIQUE,     -- тіркеу нөміріндегі код (СИЛ, ВЕД …)
    name             TEXT    NOT NULL,
    category         TEXT    NOT NULL
                     CHECK (category IN ('оқу-әдістемелік', 'аттестаттау',
                                         'контингент', 'нормативтік')),
    retention_years  INTEGER CHECK (retention_years > 0)   -- сақтау мерзімі, жыл
);

-- 3. Құжаттар — жүйенің негізгі кестесі (тіркеу журналы)
CREATE TABLE documents (
    id             INTEGER PRIMARY KEY,
    reg_number     TEXT    NOT NULL UNIQUE,       -- тіркеу нөмірі, мысалы СИЛ-2026/001
    type_id        INTEGER NOT NULL REFERENCES document_types(id) ON DELETE RESTRICT,
    title          TEXT    NOT NULL,
    doc_date       TEXT    NOT NULL CHECK (doc_date LIKE '____-__-__'),           -- ЖЖЖЖ-АА-
KK
    academic_year  TEXT    NOT NULL CHECK (academic_year LIKE '____-____'),       -- 2026-2027
    department_id  INTEGER NOT NULL REFERENCES departments(id) ON DELETE RESTRICT,
    author         TEXT,                          -- жауапты қызметкер
    status         TEXT    NOT NULL DEFAULT 'жоба'
                   CHECK (status IN ('жоба', 'келісуде', 'бекітілді', 'мұрағатта')),
    pages          INTEGER CHECK (pages > 0)
);

-- 4. Білім алушылар
CREATE TABLE students (
    id           INTEGER PRIMARY KEY,
    record_book  TEXT    NOT NULL UNIQUE,         -- ID нөмірі
    full_name    TEXT    NOT NULL,
    group_name   TEXT    NOT NULL,                -- академиялық топ, мысалы ЖИТ-25-1
    course       INTEGER NOT NULL CHECK (course BETWEEN 1 AND 4),
    email        TEXT    UNIQUE
);

-- 5. Құжат пен білім алушы арасындағы «көпке-көп» байланыс
--    (бір бұйрықта бірнеше білім алушы, бір білім алушыда бірнеше құжат)
CREATE TABLE document_students (
    document_id  INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
    student_id   INTEGER NOT NULL REFERENCES students(id)  ON DELETE CASCADE,
    note         TEXT,                            -- ескерту
    PRIMARY KEY (document_id, student_id)
);

-- 6. Қосымшаның пайдаланушылары (жүйеге кіру үшін)
CREATE TABLE users (
    id             INTEGER PRIMARY KEY,
    username       TEXT    NOT NULL UNIQUE,
    password_hash  TEXT    NOT NULL,              -- құпиясөз ашық сақталмайды
    role           TEXT    NOT NULL CHECK (role IN ('admin', 'visitor')),
    full_name      TEXT
);

-- Іздеуді жылдамдататын индекстер
CREATE INDEX idx_documents_date   ON documents(doc_date);
CREATE INDEX idx_documents_status ON documents(status);
```

```
-- Көрініс: тіркеу журналы (үш кесте біріктірілген)
CREATE VIEW v_registry AS
SELECT d.id, d.reg_number, d.doc_date, t.code AS type_code, t.name AS doc_type,
       t.category, d.title, dep.short_name AS department, d.author,
       d.status, d.academic_year, d.pages
  FROM documents d
  JOIN document_types t ON t.id = d.type_id
  JOIN departments dep  ON dep.id = d.department_id;

-- Триггер: бекітілген немесе мұрағаттағы құжатты жоюға тыйым салады
CREATE TRIGGER trg_protect_approved
BEFORE DELETE ON documents
WHEN OLD.status IN ('бекітілді', 'мұрағатта')
BEGIN
    SELECT RAISE(ABORT, 'Бекітілген немесе мұрағаттағы құжатты жоюға болмайды');
END;
```

# ҚОСЫМША Ә
## Бағдарлама кодының негізгі фрагменттері

*database.py – базаға қосылу және қазақ әліпбиі бойынша салыстыру*

```
KAZAKH_ALPHABET = "аәбвгғдеёжзийкқлмнңоөпрстуұүфхһцчшщъыіьэюя"
_KZ_ORDER = {ch: i for i, ch in enumerate(KAZAKH_ALPHABET)}


def kazakh_key(text):
    """Мәтінді қазақ әліпбиінің реті бойынша салыстыруға арналған кілт."""
    return [(_KZ_ORDER.get(ch, -1), ch) for ch in (text or "").lower()]


def kazakh_collation(a, b):
    """SQLite үшін KAZAKH салыстыру ережесі: -1, 0 немесе 1 қайтарады."""
    ka, kb = kazakh_key(a), kazakh_key(b)
    return (ka > kb) - (ka < kb)


def register_kazakh(conn):
    """kz_lower() функциясы мен KAZAKH салыстыру ережесін қосылымға тіркейді."""
    conn.create_function("kz_lower", 1, lambda s: s.lower() if isinstance(s, str) else s)
    conn.create_collation("KAZAKH", kazakh_collation)


# ---------------------------------------------------------------------------
#  Қосылу және базаны құру
# ---------------------------------------------------------------------------
def connect(db_path=DB_PATH):
    """Базаға қосылып, сыртқы кілттерді тексеруді және қазақша сұрыптауды қосады."""
    conn = sqlite3.connect(db_path)
    conn.execute("PRAGMA foreign_keys = ON")
    register_kazakh(conn)
    return conn
```

*auth.py – құпиясөзді хэштеу және жүйеге кіру*

```
def hash_password(password):
    """Құпиясөздің хэші: 'pbkdf2_sha256$итерация$тұз$хэш' пішімінде."""
    salt = secrets.token_hex(16)
    digest = hashlib.pbkdf2_hmac("sha256", password.encode(), salt.encode(), ITERATIONS)
    return f"pbkdf2_sha256${ITERATIONS}${salt}${digest.hex()}"


def verify_password(password, stored):
    """Енгізілген құпиясөздің хэшін қайта есептеп, сақталған хэшпен салыстырады."""
    try:
        _, iterations, salt, expected = stored.split("$")
    except ValueError:
        return False
    digest = hashlib.pbkdf2_hmac("sha256", password.encode(), salt.encode(), int(iterations))
    return hmac.compare_digest(digest.hex(), expected)


def authenticate(conn, username, password):
    """Логин мен құпиясөз дұрыс болса пайдаланушыны, әйтпесе None қайтарады."""
    row = conn.execute(
        "SELECT id, username, password_hash, role, full_name FROM users WHERE username = ?",
        ((username or "").strip(),)).fetchone()
    if row is None or not verify_password(password or "", row[2]):
        return None
    return {"id": row[0], "username": row[1], "role": row[3], "full_name": row[4]}
```

*repository.py – тізілімді сүзгілеу және құжатты сақтау*

```
    # ----- құжаттар тізілімі: барлық сүзгілер WHERE шартына айналады ----------
    WHERE = """ WHERE (:q = '' OR kz_lower(title) LIKE '%' || kz_lower(:q) || '%'
                               OR kz_lower(reg_number) LIKE '%' || kz_lower(:q) || '%')
                  AND (:category = '' OR category = :category)
                  AND (:type = '' OR type_code = :type)
                  AND (:year = '' OR academic_year = :year)
```

```
                  AND (:dep = '' OR department = :dep)
                  AND (:public = 0 OR status IN ('бекітілді', 'мұрағатта'))"""

    def _params(self, f):
        return {"q": (f.get("q") or "").strip(), "category": f.get("category") or "",
                "type": f.get("type") or "", "year": f.get("year") or "",
                "dep": f.get("dep") or "", "status": f.get("status") or "",
                "public": 0 if self.is_admin else 1}

    def registry(self, f):
        return self.query(
            "SELECT id, reg_number, doc_date, doc_type, category, title, department, author,
status "
            "FROM v_registry" + self.WHERE + " AND (:status = '' OR status = :status) "
            "ORDER BY doc_date DESC, reg_number DESC", self._params(f))

    def status_counts(self, f):
        """Мәртебе батырмаларындағы сандар (GROUP BY)."""
        return {r["status"]: r["n"] for r in self.query(
            "SELECT status, COUNT(*) AS n FROM v_registry" + self.WHERE + " GROUP BY status",
            self._params(f))}
    ...
    def save_document(self, data, doc_id=None):
        self.require_admin()
        values = [data.get(f) for f in self.FIELDS]
        with self.conn:                                  # транзакция
            if doc_id is None:
                cur = self.conn.execute(
                    f"INSERT INTO documents ({', '.join(self.FIELDS)}) "
                    f"VALUES ({', '.join('?' * len(self.FIELDS))})", values)
                return cur.lastrowid
            self.conn.execute(
                f"UPDATE documents SET {', '.join(f + ' = ?' for f in self.FIELDS)} WHERE id =
?",
                values + [doc_id])
            return doc_id

    def set_status(self, doc_id, status):
        self.require_admin()
        with self.conn:
            self.conn.execute("UPDATE documents SET status = ? WHERE id = ?", (status,
doc_id))

    def delete_document(self, doc_id):
        self.require_admin()
        with self.conn:                                  # бекітілген құжатты триггер қорғайды
            self.conn.execute("DELETE FROM documents WHERE id = ?", (doc_id,))
```

# ҚОСЫМША Б
# step04_update_delete.py скриптінің толық мәтіні

```
"""
4-қадам. Деректерді өзгерту (UPDATE) және жою (DELETE).

Жаңа құжат тіркеліп, мәртебесі өзгертіледі, содан кейін жойылады.
Сыртқы кілттердің CASCADE және RESTRICT ережелері мен триггердің
жұмысы да тексеріледі.
"""
import sqlite3

from database import DB_PATH, print_rows

print("=" * 64)
print("4-ҚАДАМ. ДЕРЕКТЕРДІ ӨЗГЕРТУ ЖӘНЕ ЖОЮ")
print("=" * 64)

conn = sqlite3.connect(DB_PATH)
conn.execute("PRAGMA foreign_keys = ON")
cur = conn.cursor()


def show(reg):
    print_rows(cur.execute(
        "SELECT reg_number, status, pages FROM documents WHERE reg_number = ?", (reg,)))


# 1. Жаңа құжат тіркеу және оған екі білім алушыны байланыстыру
cur.execute(
    """INSERT INTO documents (reg_number, type_id, title, doc_date, academic_year,
                              department_id, author)
       VALUES ('БК-2026/005', 6, 'Білім алушыларды басқа топқа ауыстыру туралы',
               '2026-10-01', '2026-2027', 4, 'Кенжебаева Ә.Н.')""")
new_id = cur.lastrowid
cur.executemany("INSERT INTO document_students VALUES (?, ?, 'ЖИТ-25-1 тобына')",
                [(new_id, 9), (new_id, 10)])
conn.commit()
print("\n▶ Жаңа құжат тіркелді (мәртебесі әдепкі бойынша «жоба»):")
show("БК-2026/005")

# 2. UPDATE: бір жазбаның мәртебесі мен беттер санын өзгерту
cur.execute("UPDATE documents SET status = 'келісуде', pages = 2 WHERE reg_number = 'БК-
2026/005'")
conn.commit()
print("\n▶ UPDATE: мәртебе «келісуде», беттер саны 2:")
show("БК-2026/005")

# 3. UPDATE бірнеше жазбаға: беттер саны жоқ жобаларға 1 бет жазылады
cur.execute("UPDATE documents SET pages = 1 WHERE status = 'жоба' AND pages IS NULL")
print(f"\n▶ UPDATE бірнеше жолға: {cur.rowcount} құжаттың беттер саны толтырылды")
conn.rollback()                       # мысал ғана: өзгеріс сақталмайды
print("  rollback(): бұл өзгеріс кері қайтарылды")

# 4. DELETE + ON DELETE CASCADE: құжат жойылғанда байланыс жолдары да жойылады
links = "SELECT COUNT(*) FROM document_students WHERE document_id = ?"
print("\n▶ DELETE: БК-2026/005 құжатын жою")
print("  Жоюға дейін байланысқан білім алушылар:", cur.execute(links, (new_id,)).fetchone()
[0])
cur.execute("DELETE FROM documents WHERE reg_number = 'БК-2026/005'")
conn.commit()
print("  Жойылған құжаттар:", cur.rowcount)
print("  Жойылғаннан кейін байланыс жолдары (CASCADE):", cur.execute(links,
(new_id,)).fetchone()[0])

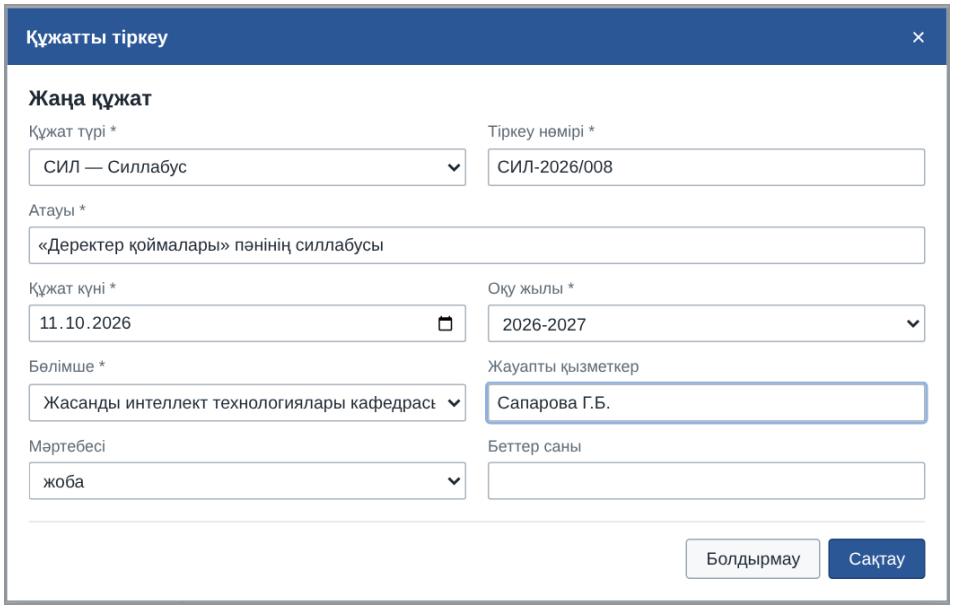
# 5. ON DELETE RESTRICT: құжаттары бар бөлімшені жоюға болмайды
print("\n▶ RESTRICT: құжаттары бар бөлімшені жою әрекеті")
try:
    cur.execute("DELETE FROM departments WHERE short_name = 'MAT'")
except sqlite3.IntegrityError as error:
    print("  ✗", error)

# 6. Триггер: бекітілген құжатты жоюға тыйым салынған
print("\n▶ Триггер: бекітілген СИЛ-2026/001 құжатын жою әрекеті")
try:
```

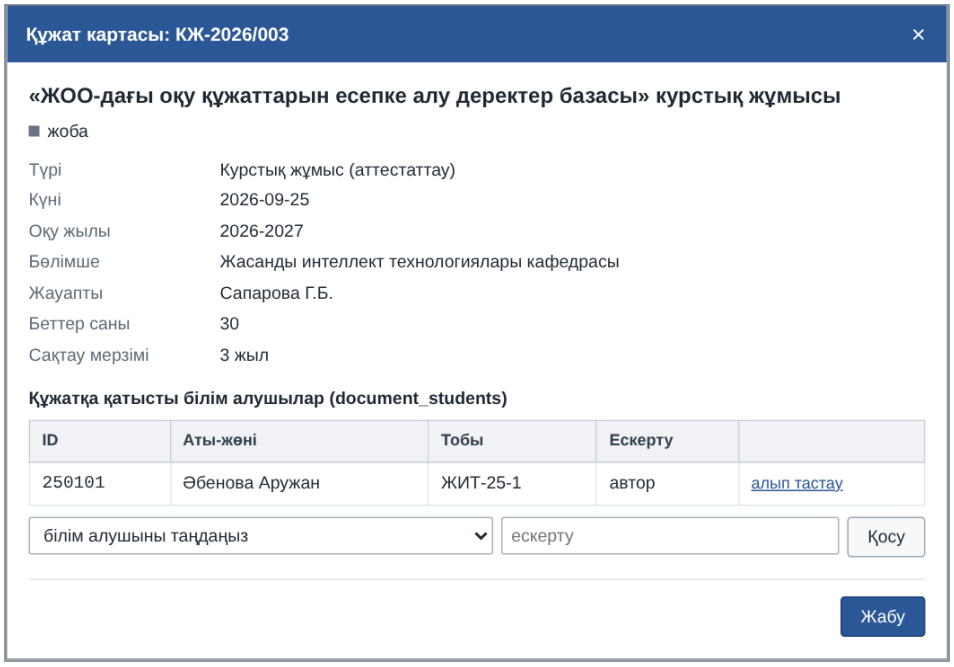
```
    cur.execute("DELETE FROM documents WHERE reg_number = 'СИЛ-2026/001'")
except sqlite3.IntegrityError as error:
    print("  ✗", error)
conn.rollback()

print("\nҚұжаттар саны:", cur.execute("SELECT COUNT(*) FROM documents").fetchone()[0])
conn.close()
print("4-қадам аяқталды.")
```

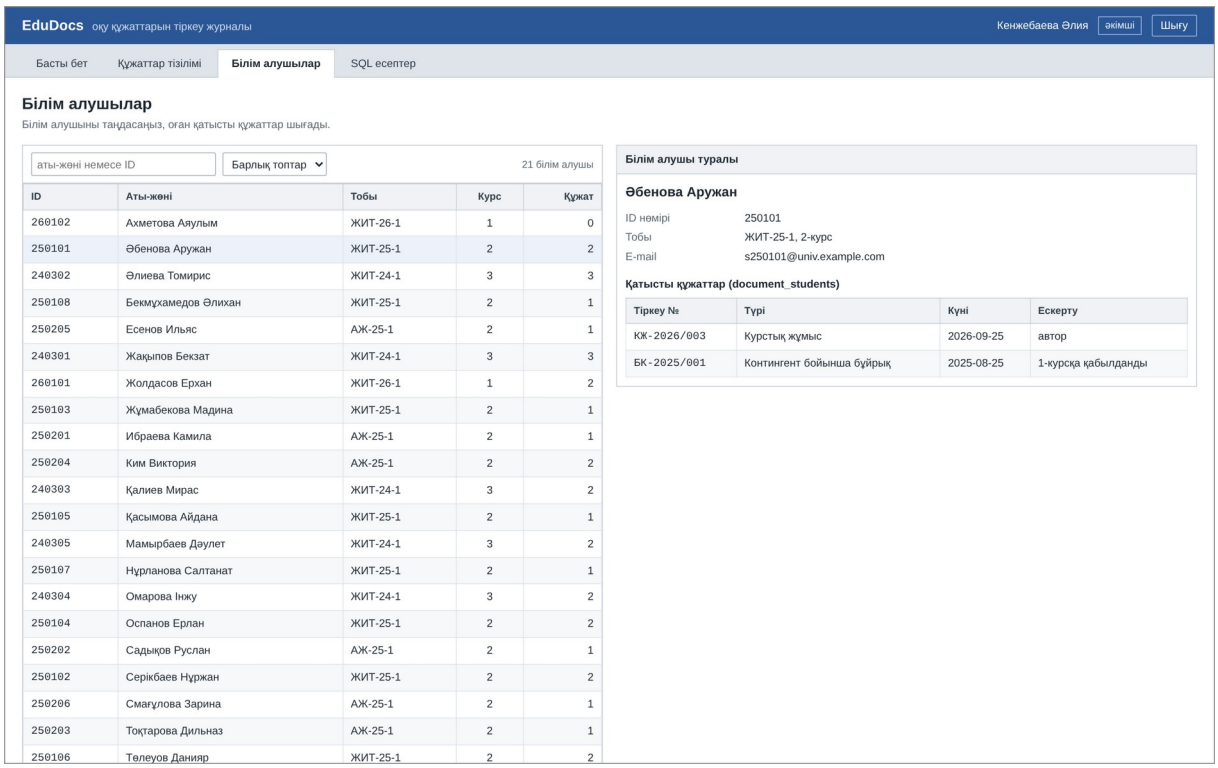
# ҚОСЫМША В
## Қосымша беттері және скрипттердің орындалу нәтижелері



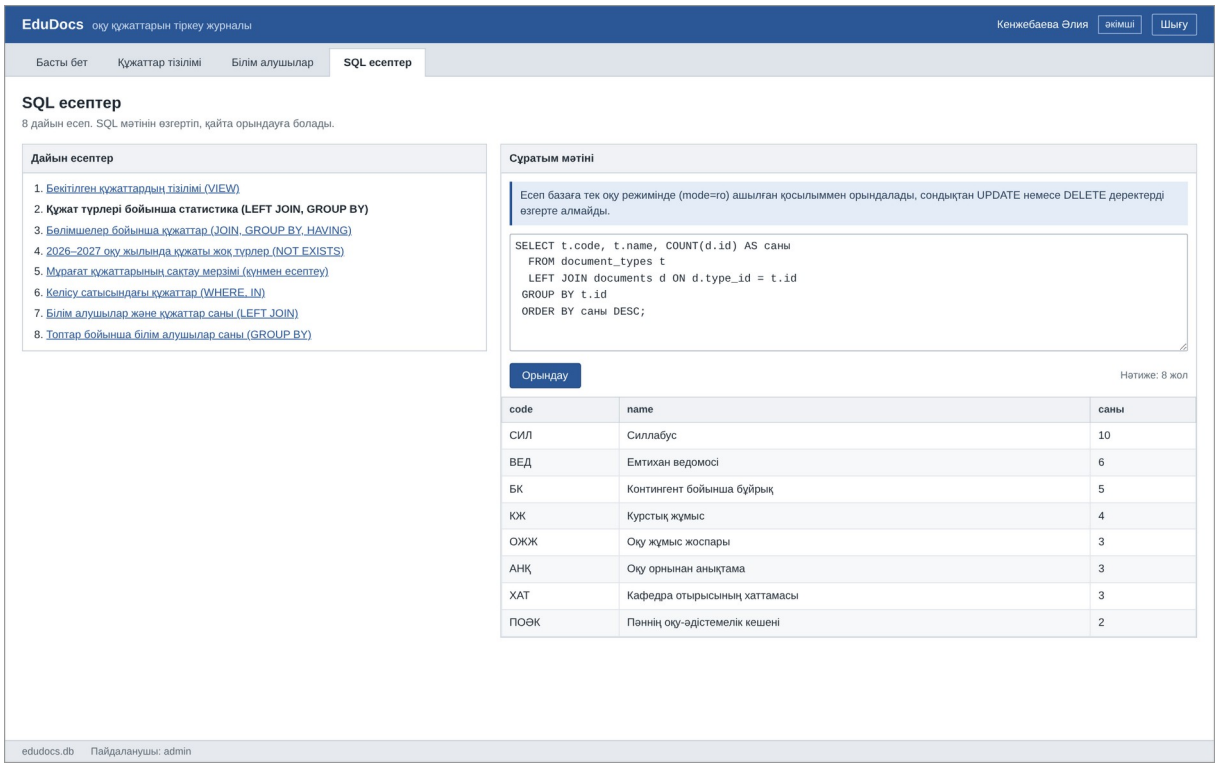

B.1-сурет. EduDocs: жаңа құжатты тіркеу формасы



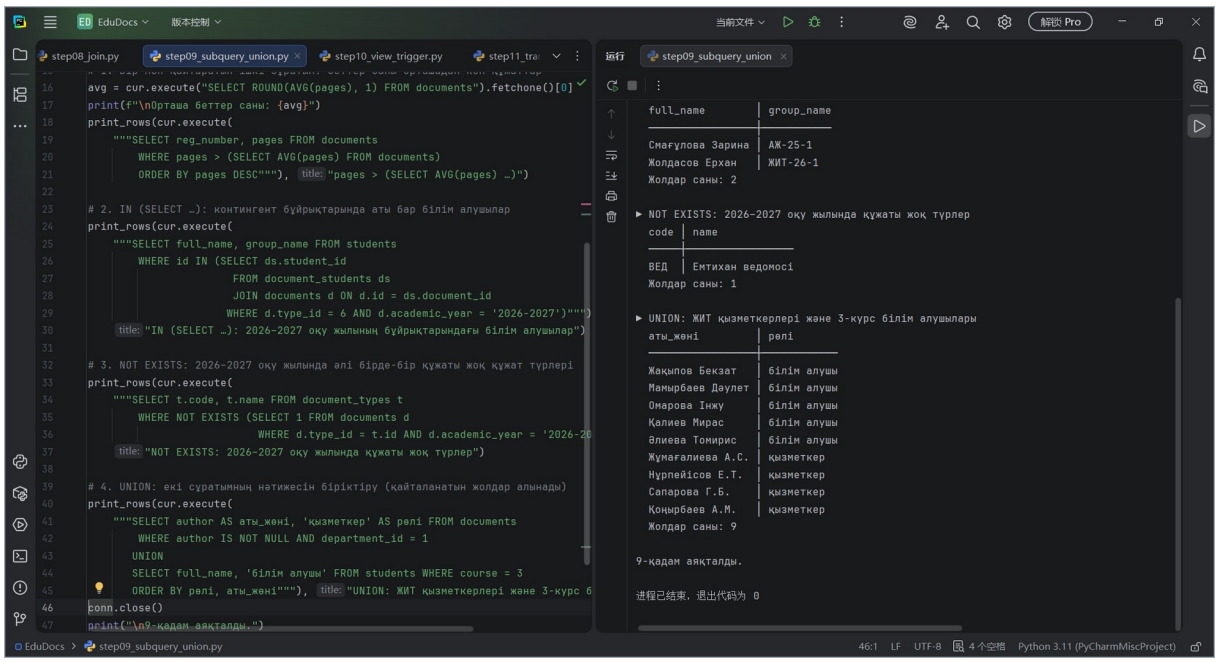

B.2-сурет. EduDocs: құжат картасы және оған қатысты білім алушылар



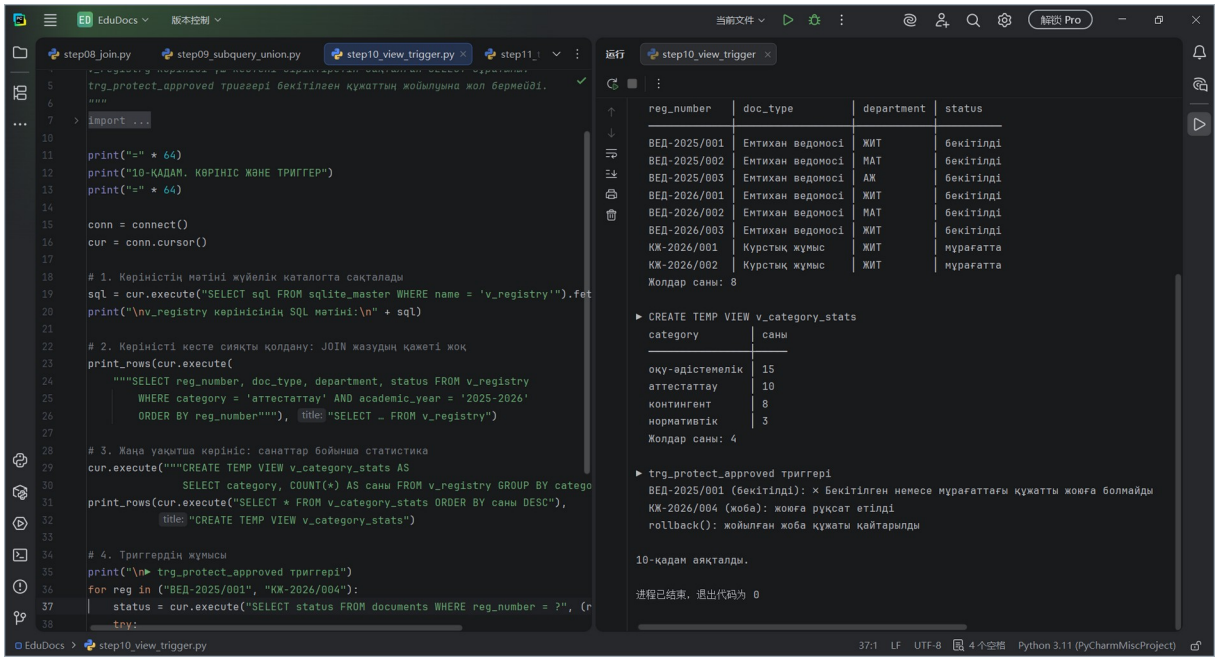

В.3-сурет. EduDocs: «Білім алушылар» беті және білім алушыға қатысты құжаттар



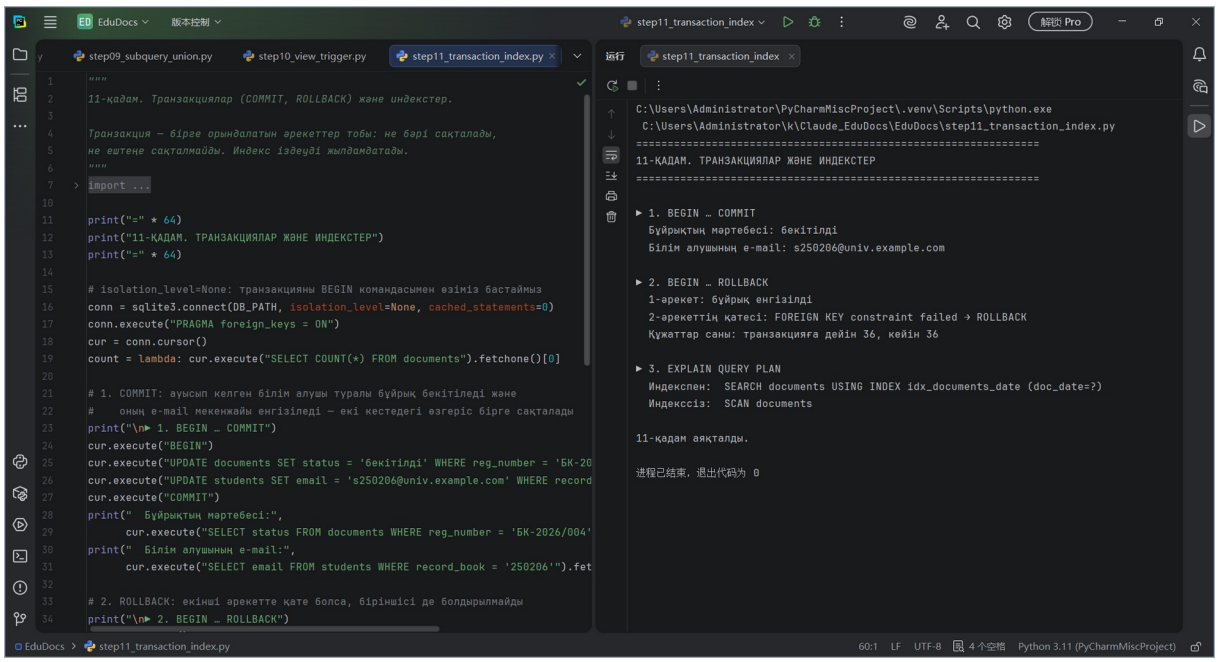

В.4-сурет. EduDocs: «SQL есептер» беті



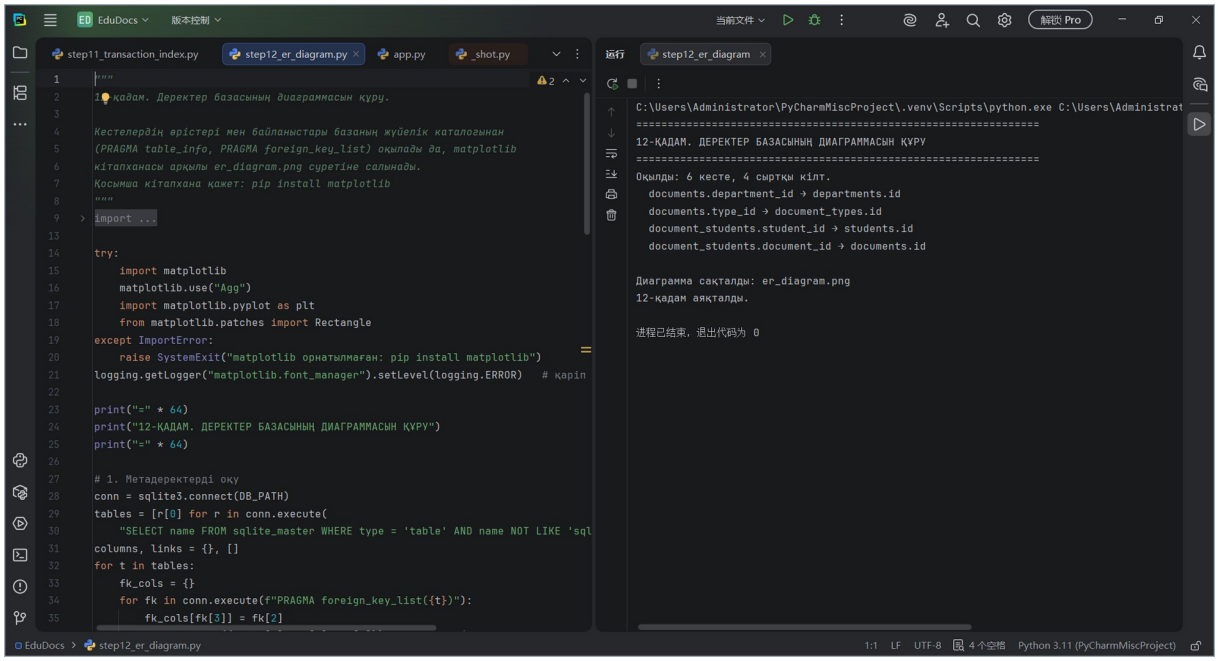

В.5-сурет. PyCharm ортасы: step09_subquery_union.py – кірістірілген сұратымдар және UNION




В.6-сурет. PyCharm ортасы: step10_view_trigger.py – көрініс және триггер




B.7-сурет. PyCharm ортасы: step11_transaction_index.py – транзакциялар және индекс




B.8-сурет. PyCharm ортасы: step12_er_diagram.py – деректер базасының диаграммасын салу

## БЕЛГІЛЕУЛЕР МЕН ҚЫСҚАРТУЛАР

Осы курстық жұмыста төмендегі белгілеулер мен қысқартулар қолданылған:

ДБ – деректер базасы;

ДББЖ – деректер базасын басқару жүйесі;

ЖОО – жоғары оқу орны;

ББ – білім беру бағдарламасы;

ПОӘК – пәннің оқу-әдістемелік кешені;

ОЖЖ – оқу жұмыс жоспары;

ҚФ – қалыпты форма (1ҚФ, 2ҚФ, 3ҚФ);

ER – Entity–Relationship, «мән–байланыс» моделі;

SQL – Structured Query Language, құрылымдалған сұратымдар тілі;

DDL – Data Definition Language, деректерді анықтау тілі;

DML – Data Manipulation Language, деректерді өзгерту тілі;

PK – Primary Key, бастапқы кілт;

FK – Foreign Key, сыртқы кілт;

API – Application Programming Interface, бағдарламалау интерфейсі;

HTTP – HyperText Transfer Protocol, гипермәтінді тасымалдау хаттамасы;

HTML, CSS – веб-беттің құрылымы мен безендірілуін сипаттайтын тілдер;

JSON – JavaScript Object Notation, деректер алмасудың мәтіндік пішімі;

PBKDF2 – Password-Based Key Derivation Function 2, құпиясөзден хэш есептеу алгоритмі;

CSV – Comma-Separated Values, мәтіндік кесте пішімі;

UTF-8 – Unicode таңбаларын кодтау пішімі.